

JAVASCRIPT

Introduction :

JavaScript (JS) is the top programming language for web development, used for both [Client-Side and Server-Side](#) purposes. It is also known as a scripting language for web [pages](#). It is commonly used to create dynamic and interactive content on websites. JS plays a crucial role in modern web browsers, enabling client-side scripting to modify web page content in real-time, enhancing user experience.

ECMAScript 2015

JavaScript was invented by Brendan Eich in 1995, and became an ECMA standard in 1997.

ECMAScript is the official name of the language.

ECMAScript versions have been abbreviated to ES1, ES2, ES3, ES5, and ES6.

Since 2016, versions are named by year (ECMAScript 2016, 2017, 2018, 2019, 2020).

ES6	ECMAScript 2015	Added let and const Added default parameter values Added Array.find() Added Array.findIndex()
------------	------------------------	--

ECMAScript 2015 was the second major revision to JavaScript 2015 is also known as ES6 and ECMAScript 6.

this chapter describes the most important features of ES6.

JAVASCRIPT

1.Values and Variables :

In JavaScript, values and variables are fundamental concepts that form the basis of programming.

Values: A value is a piece of information that a program can work with. It can be a number, text, true/false, or more complex data.

Variables: A variable is a container that holds a value. It has a name and can be used to store and manipulate data in a program.

```
console.log('welcome to variables and values page of lectures')
```

```
var myAge = 30;  
console.log(myAge)
```

NOTE* Variables can be declared using many different ways :

```
//Valid variable with ( _ in middle)  
var my_firstName = "John";  
console.log(my_firstName)
```

```
//Valid variable with ( _ as start and $)  
var _myLastName$ = "Doe";  
console.log(_myLastName$)
```

```
//Valid variable with $ symbol  
var $cityName = "New York";  
console.log($cityName)
```

```
//Invalid Variable start with Number  
var 123myAge = 25;  
console.log(123myAge)
```

```
//Invalid Variable with @ symbol  
var my@Email = "john@example.com";  
console.log(my@Email)
```

LET AND CONST - EcmaScript 2015

JAVASCRIPT

2. Data Types :

Data types define the type of values that a variable can hold.

1. Types of Primitive Data types :

1. **Number:** Represents numeric values, including integers and floating-point numbers.

Example :	<code>var myFavNum = 100;</code>	Output
	<code>console.log(myFavNum);</code>	100
	<code>console.log(typeof myFavNum);</code>	number

2. **String:** Represents a sequence of characters enclosed in single or double quotes.

Example:	<code>var myName = "vinod";</code>	Output
	<code>console.log(myName);</code>	vinod
	<code>console.log(typeof myName);</code>	string

3. **Boolean:** Represents a logical entity with two values: true or false.

Example:	<code>var isRaining = false;</code>	Output
	<code>console.log(isRaining);</code>	false
	<code>var areYouready = true;</code>	true
	<code>console.log(areYouready);</code>	boolean
	<code>console.log(typeof areYouready);</code>	

4. **Undefined:** Represents the absence of a value or an uninitialized variable.

Example:	<code>var vinod;</code>	Output
	<code>console.log(vinod);</code>	undefined
	<code>console.log(typeof vinod);</code>	undefined

5. **Null:** Represents the absence of a value. It is often used to explicitly indicate that a variable or object property has no assigned value.

Example:	<code>var badMemories = null;</code>	Output
	<code>console.log(badMemories);</code>	null

JAVASCRIPT

`console.log(type of badMemories);` `object*`

6. **BigInt**: Represents integers of arbitrary precision (available since ECMAScript 2020).

Example: `const bigNumber =`

`1234567890123456789012345678901234567890n;`

`console.log(bigNumber);`

Output : `1234567890123456789012345678901234567890n;`

7. **Symbol**: Represents a unique and immutable data type, often used to create unique identifiers.

Example: `const mySymbol = Symbol("description");`

`console.log(mySymbol);`

Output : `Symbol(description)`

2. Types of Non Primitive Data types :

1. Object
2. Array
3. Function

2.1 Data Types Interview Questions :

1: What is the difference between null and undefined in JavaScript ?

- Undefined means a variable has been declared but has yet not been assigned a value.
- Null is an assignment value. It can be assigned to a variable as a representation of no value.

2: What is the purpose of typeof operator in JavaScript ?

- You can use the typeof operator to find the data type of a JavaScript variable.

5: Convert a string to a number?

JAVASCRIPT

- We just need to add the '+' sign before the string or by number constructor

- Example:

```
var myNaam = "100";  
console.log(typeof +myNaam);  
console.log(typeof Number(myNaam));
```

Output

number
number

6: Convert a number to a string?

- We just need to add an empty string after the number or by string constructor

- Example:

```
var myNum = 5;  
console.log(typeof (myNum + " "));  
console.log(typeof String(myNum));
```

Output

string
string

7: Explain the concept of truthy and falsy values in JavaScript. Provide examples. ? To check if a non-empty string is truthy or falsy in JavaScript, we can directly use if statement.

```
var name = "mohit";  
if(name){  
    console.log("truthy value");  
}  
else{  
    console.log("falsy value");  
}
```

Output

truthy value

2.2 parseInt & parseFloat Section

- **parseInt** and **parseFloat** are both functions in JavaScript used for converting strings to numbers, but they have different use cases.
- **parseInt: Definition:** parseInt is used to parse a string and convert it to an integer (whole number).

Examples:

```
const myString = "42";  
const myNumber = parseInt(myString, 10);  
console.log(myNumber);
```

Output: 42

```
const myString = "42.5";
```

JAVASCRIPT

```
const myNumber = parseInt(myString);  
console.log(myNumber);
```

Output: 42

- **parseFloat: Definition:** parseFloat is used to parse a string and convert it to a floating-point number (decimal number).

Examples:

```
const myString = "42.5";  
const myNumber = parseFloat(myString);  
console.log(myNumber);
```

Output: 42.5

TODO Key Differences:

- parseInt is used for converting to integers and ignores anything after the decimal point.
- parseFloat is used for converting to floating-point numbers, preserving the decimal part.
- Both functions will attempt to convert as much of the string as possible until an invalid character is encountered.

Here are more examples:

Output

```
console.log(parseInt("123"));  
123 (default base-10)
```

123

```
console.log(parseInt("123", 10));  
123 (explicitly specify base-10)
```

123

```
console.log(parseInt(" 123 "));  
123 (whitespace is ignored)
```

123

```
console.log(parseInt("077"));  
77 (leading zeros are ignored)
```

77

```
console.log(parseInt("1.9"));  
1 (decimal part is truncated)
```

1

JAVASCRIPT

```
console.log(parseFloat("1.9"));  
1.9 (process after decimal)
```

1.9

When we will not get an Output

```
console.log(parseInt("&123"));  
console.log(parseInt("-123"));  
console.log(parseInt("xyz"));  
NaN (input can't be converted to an integer)
```

Output : NaN
Output : -123
Output : NaN

What is the purpose of the NaN value in JavaScript ?

In JavaScript NaN is short for "Not-a-Number". The isNaN() method returns true if a value is NaN. //The isNaN() method converts the value to a number before testing it.

```
console.log(isNaN('2'));      Output : false
```

3. Concat / coercion

Concatenation in JavaScript

- In JavaScript, the + sign is not only used for arithmetic addition but also for string concatenation. When the + operator is used with strings, it concatenates the strings together.
- It's important to note that if any operand of the + operator is a string, JavaScript will treat the other operands as strings as well, resulting in string concatenation. If both operands are numbers, the + operator performs numeric addition.

Example:

```
const str = "Hello" + " " + "World";  
console.log(str);
```

Output : Hello World

JAVASCRIPT

```
const s1 = "myRollno " + 101;  
console.log(s1);
```

Output : myRollno 101

Type coercion

- Type coercion refers to **the automatic conversion of values from one data type to another, typically performed during operations or comparisons involving different data types**
- Type coercion is the automatic conversion of "values" from one data type to another.
- It is a fundamental part of JavaScript and can be used to make code more readable and efficient.
- There are two types of coercion in JavaScript: **implicit and explicit**. Implicit coercion happens automatically, while explicit coercion is done manually by the programmer.
- It's worth noting that type coercion can lead to unexpected results, so it's essential to be aware of how JavaScript handles these situations.

Implicit conversion

- ```
let sum = "5" + 10; //here 10 is converted into string
console.log(sum);
console.log(typeof sum);
```

Output : 510  
Output : string
- ```
let result = '3' - 2;  
console.log(result);  
console.log(typeof result);
```

Output : 1
Output : Number
- ```
let result = '3' - 'hello';
console.log(result);
console.log(typeof result);
```

Output : NaN  
Output : Number
- ```
let result = '3' - true; // true is convert into 1 and '3' into 3  
console.log(result);
```

Output : 2

JAVASCRIPT

`console.log(typeof result);`

Output : Number

- `let result = '3' - Null;`
`console.log(result);`
`console.log(typeof result);`

Output : 3

Output : Number

Explicit conversion

- `let result = '3';`
`result = Number(result);`
`console.log(typeof result);`

Output : Number

- `let result = null;`
`result = Number(result);`
`console.log(result);`
`console.log(typeof result);`

Output : 0

Output : Number

- `let result = null;`
`result = String(result);`
`console.log(result);`
`console.log(typeof result);`

Output : Null

Output : String

- `let result = 20.00;`
`result = parseInt(result);`
`console.log(result);`
`console.log(typeof result);`

Output : 20

Output : Number

4. Expressions & Operators

1st we will see what the expression means and also what is the operand and operator in any expression

5 + 10 --- expression

5,10 --- operand

Manmohan Choudhary

JAVASCRIPT

+ --- operator

Types of Operators in JS

- Assignment operators
- Arithmetic operators
- In arithmetic we increment and decrement operators.
- Comparison operators
- Logical operators
- String operators
- Conditional (ternary) operator

1: Assignment operators : Assignment operators in programming are symbols used to assign values to variables. They take the value on the right side of the operator and assign it to the variable on the left side.

Examples

```
var myFavNum = 100;  
console.log(myFavNum);
```

Output
100

2: Arithmetic operators : Arithmetic operators in programming perform basic mathematical operations on variables or values. They include addition, subtraction, multiplication, division, and modulus.

- **Addition (+)**: Adds two values or variables.

Example:

```
let x = 5;  
let y = 10;  
let sum = x + y;  
console.log(sum);
```

Output
15

- **Subtraction (-)**: Subtracts the right operand from the left operand.

Example:

```
let a = 10;  
let b = 7;  
let difference = a - b;
```

Output
3

JAVASCRIPT

`console.log(difference);`

- **Multiplication (*)**: Multiplies two values or variables.

Example:

```
let p = 4;  
let q = 6;  
let product = p * q;  
console.log(product);
```

Output
24

- **Division (/)**: Divides the left operand by the right operand.

Example:

```
let m = 15;  
let n = 3;  
let quotient = m / n;  
console.log(quotient);
```

Output
5

- **Modulus (%)**: Returns the remainder when the left operand is divided by the right operand.

Example:

```
let c = 17;  
let d = 5; let remainder = c % d;  
console.log(remainder);
```

Output
2

- **(**)** : Returns the result by multiplying the number by the power.

Example:

```
Let a = 10;  
Let power = 5;  
console.log(a**power);
```

Output

100000

- In arithmetic we increment and decrement operators.

1. **Postfix Increment** : `let a=1;`

```
console.log(a++);  
console.log(a);
```

Output
1
2

JAVASCRIPT

- | | |
|--|------------------|
| 2. Prefix Increment : <code>let a=1;</code>
<code>console.log(++a);</code>
<code>console.log(a);</code> | Output
2
2 |
| 3. Postfix decrement : <code>let a=1;</code>
<code>console.log(a--);</code>
<code>console.log(a);</code> | Output
1
0 |
| 4. Prefix decrement : <code>let a=1;</code>
<code>console.log(--a);</code>
<code>console.log(a);</code> | Output
0
0 |

2.1 : InterView Question

- | | |
|--|----------------|
| • <code>let result = "hello" / 2;</code>
<code>console.log(result);</code> | Output
NaN |
| • <code>var result = 0.1 + 0.2;</code>
<code>console.log(result.toFixed(2));</code> | Output
0.30 |
| • <code>var r = 55 * "hello";</code>
<code>console.log(r);</code> | Output
NaN |

3: String Operators :

String Operators Examples

<code>Var str1 = "Hello";</code> <code>var str2 = "World";</code> <code>var str3 = str1 + " " + str2;</code> <code>console.log(str3);</code>	Output HelloWorld
---	----------------------

<code>var s='10';</code> <code>var t='20';</code> <code>console.log(s+t);</code>	Output 1020
--	----------------

JAVASCRIPT

3.1 InterView Question

- `console.log("5" + 3);` // **Outputs "53"**
- `console.log("5" + 3);` //In case + operator in one str and one num it concatenate.
- `console.log("5" / 3);` //In case -, *, /, % operator in one str and one num it typecast the str and calculate the values
Output : 1.6666666666666667

4: comparison operators

Comparison operators in JavaScript are used to compare values and return a Boolean result (true or false).

- **Equal (==):** Checks if two values are equal, **performing type coercion** if necessary.

Example :

```
console.log(5 == 5);  
console.log(5 == '5');
```

Output

True
True

- **Strict Equal (===):** Checks if two values are equal **without performing type coercion**.if also check the data type of values.

Example :

- `console.log(5 === "5");`
- `console.log(5 === 5);`

Output

False
True

JAVASCRIPT

- **Not Equal (!= 👉 !=):** Checks if two values are not equal, performing type coercion if necessary.

Example :

- console.log(5 != 5);
- console.log(5 != '5');

Output

False
False

- **Strict Not Equal (!==):** Checks if two values are not equal **without performing type coercion**.if also checks the data type of values.

Example :

- let a = true;
- let b = 1;
- console.log(a !== b);
- console.log(a != b);

Output

True
False

- **Greater Than (>):** Checks if the value on the left is greater than the value on the right.

Example: 10 > 5 evaluates to true.

- console.log(5 > 2);
- console.log(10 > '10');
- console.log("10" > 10);

Output

True
False
False

- **Less Than (<):** Checks if the value on the left is less than the value on the right.

Example: 5 < 10 evaluates to true.

- console.log(5 < 10);
- console.log('10' < 5);
- console.log(5 < '10');

Output

True
False
True

- **Greater Than or Equal To (>=):** Checks if the value on the left is greater than or equal to the value on the right.

Example: 10 >= 10 evaluates to true.

- console.log(10 >= 10);
- console.log('10' >= 10);
- console.log(10 >= '10');

Output

True
True
True

- **Less Than or Equal To (<=):** Checks if the value on the left is less than or equal to the value on the right.

Example: 5 <= 10 evaluates to true.

- console.log(5 <= 10);
- console.log('5' <= 10);

Output

True
True

JAVASCRIPT

- `console.log(5 <= '10');` True

4.1 InterView Question

What is the difference between == and === operators in JavaScript ?

- The equality == operator is a comparison operator that compares two values and returns true if they are equal. **(performing type coercion)**
- The strict equality === operator is a comparison operator, but it compares two values and returns true only if they are equal and of the same type. **(without performing type coercion)**

- **Example :**

```
var num1 = 1;  
var num2 = '1';  
if(num1 === num2){  
  console.log("equal"); }  
else{  
  console.log("not equal"); }
```

Output

not equal

5: Logical operators in JavaScript

There are three main logical operators: && (logical AND), || (logical OR), and ! (logical NOT).

LOGICAL OPERATORS

A	B	A B	A && B	!A
False	False	False	False	True
True	False	True	False	False
False	True	True	False	True
True	True	True	True	False

- **Logical AND (&&):** Returns true if both operands are true, otherwise, it returns false.

JAVASCRIPT

- **Example:**

```
var x1 = 5;  
var y1 = 10;  
console.log(x1 > 0 && y1 > 0);
```

Output

True

- **Logical OR (||):** Returns true if at least one of the operands is true, otherwise, it returns false.

- **Example:**

```
var a1 = 15;  
var b1 = 0;  
console.log(a1 > 10 || b1 > 10);
```

Output

True

- **Logical NOT (!):** Returns true if the operand is false, and false if the operand is true.

- **Example:**

```
var isOpen = false;  
console.log(!isOpen);
```

Output

True

5.1 InterView Question

Combining logical operators allows you to create complex conditions:

Example : Write a program that determines if a person is eligible to drive based on their age being greater than or equal to 18 and having a valid driver's licence

Example:

```
var age = 18;  
var hadDrivingLicense = true;  
if(age >= 18 && hadDrivingLicense == true){  
  console.log("drive");  
}  
else{  
  console.log("don't drive");  
}
```

Output

drive

How would the result change if hasDriverLicense was set to false.

Example:

```
var age = 18;  
var hadDrivingLicense = false;
```

Output

don't drive

JAVASCRIPT

```
if(age >= 18 && hadDrivingLicense == true){  
  console.log("drive"); }  
else{  
  console.log("don't drive"); }
```

6: Conditional (ternary) operator:

syntax: condition ? expressionIfTrue : expressionIfFalse;

Example : write a program to check if the candidate isEligibleToDrive or not?
Age must be equal to or greater than 18.

```
var candidates = 12;  
var ans = candidates >= 18 ? "true" : "false";  
console.log(ans);
```

Output
False

Example : Let's say you have a variable score representing a student's exam score. If the score is greater than or equal to 60, the student passes; otherwise, they fail. Use the conditional (ternary) operator to determine the result and store it in a variable called result. Log the result to the console

```
var score = 70;  
var result = score >= 60 ? "passed" : "failed";  
console.log(result);
```

Output
passed

JAVASCRIPT

6.1 Combined Interview Questions

- `console.log("5" - 3);`
- `console.log(2 < 12 < 5);`
- `console.log("20" + 10 + 10);`

Output

2

True

201010

5. If-Else/Loops (Conditional Statement)

If-Else statement: The if...else statement executes a statement if a specified condition is truthy. If the condition is falsy, another statement in the optional else clause will be executed.

Syntax

```
if (condition) {  
  // Code to be executed if the condition is true  
}  
else {  
  // Code to be executed if the condition is false  
}
```

Example :

```
var temp = 25;  
if (temp >= 30)  
{  
  console.log("let's go to beach");  
}  
else if (temp > 20 && temp < 30)
```

Output

Cool

JAVASCRIPT

```
{  
  console.log("cool");  
}  
else  
{  
  console.log("watch tv ");  
}
```

Note* We can also use an else if clause to check additional conditions.

Interview Questions :

1: Write a program to check if a number is even or odd.

```
var value = 1;  
if (value % 2 == 0)  
{  
  console.log("even number");  
}  
else{  
  console.log("odd number");  
}
```

Output

odd number

2: Write a program to check if a number is prime.

```
var prime = 11;  
var f = 0;  
for (var i = 1; i <= prime; i++) {  
  if(prime % i == 0) {  
    f++;  
  }  
}  
if (f > 2) {
```

Output

number is prime

JAVASCRIPT

```
console.log("number is not prime");  
}  
else {  
  console.log("number is prime");  
}
```

OR

```
var n = 11;  
var isprime = true;  
for (let j = 2; j < n; j++) {  
  if (n % j == 0) {  
    isprime = false; break;  
  }  
}  
if (isprime == true) {  
  console.log("prime number");  
} else { console.log("not a prime number");}
```

Output

prime number

Note * Prime numbers are numbers that have only 2 factors: 1 and themselves.

All prime numbers greater than 2 are odd.

However, not all odd numbers are prime.

3: Write a program to check if a number is positive, negative, or zero.

```
var x = 1;  
if (x == 0) {  
  console.log("number is zero");  
}  
else if (x > 0) {  
  console.log("number is positive");  
}  
else {  
  console.log("number is negative");  
}
```

Output

number is positive

Switch Statement :

Switch Statement: The switch statement is used to perform different actions based on different conditions.

JAVASCRIPT

Syntax:

```
switch (expression) {  
  case value1:  
    // Code to be executed if expression === value1  
    break;  
  case value2:  
    // Code to be executed if expression === value2  
    break;  
  // More cases can be added as needed  
  default:  
    //Code to be executed if none of the cases match  
    break;  
}
```

Example:

```
var day = "Saturday";  
switch (day) {  
  case "Monday":  
    console.log("good morning in office");  
    break;  
  case "Wednesday":  
    console.log("movie time");  
    break;  
  case "Sunday":  
    console.log("just sleep");  
    break;  
  default:  
    console.log("good keep coding");  
    break;  
}
```

Output

good keep coding

Loop in js:

Loops can execute a block of code a number of times.

1. **While Loop** :A while loop in JavaScript is a control structure that repeatedly executes a block of code as long as a specified condition remains true. The loop continues iterating while the condition is true, and it terminates when the condition becomes false. \

JAVASCRIPT

Syntax: while (condition) {

```
// Code to be executed as long as the condition is true
}
```

Example : Simple while loop to count from 1 to 10.

```
var e = 1;
while (e <= 10)
{
  console.log(e);
  //console.log(5*e);
  e++;}
```

Output

1 2 3 4 5 6 7 8 9 10

2. **Do...While Loop:** A do...while loop in JavaScript is similar to a while loop, but it guarantees that the loop body will be executed at least once before checking the loop condition. The loop continues to execute while the specified condition is true, and it terminates when the condition becomes false.

Syntax: do {

```
//Code to be executed at least once
} while (condition);
```

Example : Simple do...while loop to count from 1 to 10.

```
var d = 1;
do {
  console.log(d);
  d++;
}while (d <= 10);
```

Output

1 2 3 4 5 6 7 8 9 10

Common Use Cases:

- When you want to guarantee the execution of the loop body at least once.
- When the number of iterations is not known beforehand, and you want to validate the condition after the first iteration.

3. **For Loop:** A for loop in JavaScript is a control flow statement that allows you to repeatedly execute a block of code a specified number of

JAVASCRIPT

times. It's particularly useful when you know the exact number of iterations needed.

Syntax: `for (initialization; condition; iteration) {
 // Code to be executed in each iteration
}`

Initialization: Executed before the loop starts. Often used to initialise a counter variable.

Condition: Evaluated before each iteration. If false, the loop terminates.

Iteration: Executed after each iteration. Typically used to update the loop control variable.

Example : Simple for loop to count from 1 to 10.

```
for (var i = 1; i <= 10; i++)  
{  
    console.log(i);  
}
```

Output

1 2 3 4 5 6 7 8 9 10

Key Point:

- The initialization, condition, and iteration expressions are optional. You can omit any or all of them, but you must include the semicolons.
- The code `for (;;) {}` represents an infinite loop in JavaScript. This construct is commonly used when you want a loop to run indefinitely or until a break statement is encountered within the loop. It's equivalent to `while (true) {}`.

Use case: Game Development:

- In game development, an infinite loop can be used to continuously update and render game frames until a specific condition (e.g., game over) is met.
- `for (;;) { // Update game logic and render frames // }`

Common Use Cases:

- When you know the exact number of iterations needed.
- Iterating over elements in an array.
- Performing a task a specific number of times.

Practice :

JAVASCRIPT

Example: Calculate the sum of numbers from 1 to 10 using a for loop.

```
var sum = 0;
for (var j = 1; j <= 10; j++){
  sum = sum + j;
}
console.log(sum);
```

Output

1 3 6 10 15 21 28 36 45 55
55

Example 3: Generating a times table of 5 with a for loop.

```
var table = 5;
for (let i = 1; i <= 10; i++) {
  console.log(table * i);
}
```

Output

5 10 15 20 25 30 35 40 45 50

More Practice

1: program To check if a year is a leap year

if a year is divisible by 4 and not divisible by 100,
or if the year is divisible by 400, then it is a leap year. otherwise, it is not a leap year.

Example:

```
var year = 2024;
if ((year % 4 === 0 && year % 100 !== 0) || year % 400 === 0) {
  console.log(year, "is a leap year"); }
else {
  console.log(year, "is not a leap year");
}
```

Output : 2024 is a leap year

In JavaScript, console.log prints each call on a new line by default

Example :

```
for (let i = 1; i <= 5; i++) {
```

Output

JAVASCRIPT

```
var pattern = "";
for (let j = 1; j <= i; j++) {
  pattern = pattern + " *";
}
console.log(pattern);
}
```

```
*
**
***
****
*****
```

6. Functions In Js

In JavaScript, a function is a block of reusable code that performs a specific task or set of tasks. Functions are used to organise code into modular and manageable pieces, promote code reuse, and make programs more readable.

Example: Three students at a time want to find the sum of two numbers.

1st student :var a = 5; b = 10; var sum0 = a + b; console.log(sum0);	Output 15
---	-------------------------

2nd student :var a2 = 50; b2 = 10; var sum1 = a1 + b2; console.log(sum1);	Output 60
--	-------------------------

3rd student : var a3 = 500; b3 = 10; var sum2= a3 + b3; console.log(sum2);	Output 510
---	--------------------------

Now perform same by the help of function

```
//function
function sum(x, y){
  return x + y;
```

JAVASCRIPT

```
}
```

```
console.log(sum(5, 10)); //function call  
console.log(sum(50,10));  
console.log(sum(500,10));
```

Output

15
60
510

Function Declaration:&

Function Statement :

- Declare a function using the function keyword, followed by the function name, parameters (if any), and the function body.
- This step defines the function and specifies what code should be executed when the function is called.

Syntax:

```
function greet() {  
  console.log("Hello Guys, Welcome to Thapa Technical JS Course ");  
}
```

```
greet(); //function call
```

Function Invocation (Calling a Function):

- After declaring a function, you can invoke or call it by using its name followed by parentheses.
- If the function has parameters, provide values (arguments) for those parameters inside the parentheses.

Syntax:

How to call a function

```
greet();
```

Example : Write a function to find the sum of two numbers.

[Manmohan Choudhary](#)

JAVASCRIPT

//Function Declaration:

```
function add() {  
  var m = 5, n = 10;  
  console.log(m + n);  
}
```

```
// Calling the function  
add();
```

Output

15

Function Parameter:

A function parameter is a variable that is listed as a part of a function declaration. It acts as a placeholder for a value that will be provided when the function is called. Parameters allow you to pass information into a function, making it more versatile and reusable.

Syntax:

```
function functionName(parameter1, parameter2, ...params) {  
  // Function body  
  // Code to be executed when the function is called }
```

```
function myfun(name) { //parameter  
  console.log("my name is ", name);  
}
```

Output

my name is mohit

```
myfun("mohit"); //argument
```

Function Argument:

A function argument is a value that you provide when you call a function. Arguments are passed into a function to fill the parameters defined in the function declaration.

syntax:

```
functionName(argument1, argument2, ...);
```

Example: we want to greet students with one same line

JAVASCRIPT

```
function greet1(name) {  
  console.log("Hello", name, "Welcome to Thapa Technical JS Course");  
}  
greet1("vinod"); //argument  
greet1("ram"); //argument
```

Output Hello", Vinod, "Welcome to Thapa Technical JS Course.
 Hello", ram, "Welcome to Thapa Technical JS Course

Example: Write a function to find the sum of two numbers with parameters.

```
function sum(a, b) {  
  console.log(a + b);  
}  
sum(10, 30);
```

Output
40

Function expressions:

A function expression is a way to define a function as part of an expression. It can be either named or anonymous. If it's named, it becomes a named function expression.

Named function expression :

```
var result = function sum(a, b) {  
  console.log(a + b);  
};
```

Output
500

```
result(200, 300);
```

Anonymous Function expression :

An anonymous function is a **function without a name**. It can be created using **either a function expression or a function declaration** without a specified name.

```
var ans = function (x, y) {  
  console.log(x * y);  
}
```

Output
25

```
ans(5, 5);
```

JAVASCRIPT

Return Keyword :

- **Return Keyword:** In JavaScript, the return statement is used within a function to specify the value that the function should produce or provide back to the code that called it.
- The return statement stops the execution of a function and sends a value back to the caller

Syntax

return expression;

Example 1: Returning a Sum of two number.

console.log("using function declaration ");

```
function subtraction(m, n) {  
  return m - n;  
}  
console.log(subtraction(10, 5));  
console.log(subtraction(30, 20));
```

Output

5
10

console.log("using named function expression");

```
const myfun1 = function sub(m, n) {  
  return m - n;  
};  
console.log(myfun1(10, 5));  
console.log(myfun1(30, 20));
```

Output

5
10

console.log("using anonymous function expression");

JAVASCRIPT

```
const myfun = function (m, n) {  
  return m - n;  
};  
console.log(myfun(10, 5));  
console.log(myfun(30, 20));
```

Output

5
10

IIFE - Immediately Invoked Function Expression :

- An IIFE or Immediately Invoked Function Expression, is a JavaScript function that is defined and executed immediately after its creation.
- It is a way to create a self-contained block of code that doesn't interfere with the surrounding code and executes immediately.
- no need to name it.

Syntax

```
(function () {  
  // code to be executed  
})();
```

Output

```
(function (g, h) {  
  console.log(g + h);  
})(4, 5);
```

9

Practice Time (IIFE with Parameters and function expression)

Example: named Iife

```
var ans2 = (function (r, t) {  
  console.log(r + t);  
  return r + t;  
})(9, 9);
```

Output

18

sum of two number is 18

```
console.log("sum of two number is " + ans2);
```

JAVASCRIPT

Revisiting anonymous function with IIFE

An anonymous function is a **function without a name**. It can be created using **either a function expression or a function declaration** without a specified name.

Creating using an anonymous function declaration with IIFE.

```
(function () {  
    console.log(" i'm a anonymous function with iife ");  
})();
```

//with parameters and arguments

```
(function (x, y) {  
    console.log("i'm a anonymous function with iife", x + y);  
})(10, 2);
```

// Iife with Arrow function

```
((x, y) => {  
    console.log(`my result is" ${x + y}`);  
})(10, 80);
```

FAT ARROW FUNCTION COVERED INTO ECMA SCRIPT es2015 NEXT :

Function inside Function :

Declaration :

```
function fun() {  
    console.log("i am fun 1");  
    function fun2() {
```

JAVASCRIPT

```
    console.log("i am fun 2");  
  }  
  fun2();//Correct way to call  
}
```

```
fun();  
// fun2(); error because fun 2 is inside fun
```

i am fun 1
i am fun 2

Lexical scope :

Lexical scope in JavaScript refers to the scope defined by the position of variables and functions in the source code. This means that the scope of a variable is determined by its location within the nested function scopes at the time of writing, not by the order of execution during runtime.

Here are the key points to understand lexical scope in JavaScript:

1. **Static Scope:** Lexical scope is sometimes referred to as static scope because it is determined at compile-time, based on the structure of the code.
2. **Function Scope:** In JavaScript, functions create a new scope. Variables defined within a function are not accessible from outside that function.
3. **Nested Functions:** Inner functions have access to variables and functions defined in their outer scope. This means that if a function is nested within another function, the inner function can access the variables and parameters of the outer function.
4. **Global Scope:** Variables declared outside any function are in the global scope and can be accessed from anywhere in the code.

Example:

```
function outerFunction() {  
  const outerVariable = 'I am outside!';  
  
  function innerFunction() {  
    console.log(outerVariable); // innerFunction is defined within  
    outerFunction and thus has access to outerVariable due to lexical scope.  
  }  
  
  innerFunction(); // calling innerFunction  
}
```

JAVASCRIPT

```
//calling outerFunction  
outerFunction();
```

Output : I am outside!

Example 1:

```
function fun1() {  
  let a = 10;  
  console.log("i am fun1 : " + a);  
  function fun2() {  
    console.log("i am fun2 : " + a); // here a is taking lexical scope from fun1  
  }  
  fun2();  
}  
fun1();
```

Output

```
i am fun1 : 10  
i am fun2 : 10
```

Example 2:

let a = 10; //global lexical scope for both function fun1 and fun2

```
function fun1() {  
  console.log("i am fun1 : " + a);  
  function fun2() {  
    console.log("i am fun2 : " + a);  
  }  
  fun2();  
}  
fun1();
```

Output

```
i am fun1 : 10  
i am fun2 : 10
```

Debugger :

The debugger statement in JavaScript is a built-in way to trigger a breakpoint in your code, allowing you to pause execution and inspect variables, the call stack, and more in the developer tools of your browser. Here's how you can use it:

Example:

```
function fun() {  
  debugger;  
  for (let i = 1; i <= 5; i++) {  
    console.log(i);  
  }  
}
```

JAVASCRIPT

```
}  
}  
fun();
```

Pure Functions :

A pure function always produces the same output for the same input.

- **Not Modifying the State** : The function does not change or rely on anything outside its scope. It works only with its input and produces an output without altering the program's state.
- **No Side Effects Example**: This function has no side effects because it doesn't affect anything outside its scope.
- **Example** :

```
function add(a, b) {  
  return a + b; // Output depends only on inputs  
}  
console.log(add(2, 3)); // Always 5
```

Impure Functions :

An impure function may produce different outputs for the same input.

- **Modifying the State**: Changing variables, data, or objects outside the function's scope or altering the program's existing state.
- **Side Effects** : Any action performed by a function that affects something outside its scope, such as: Modifying global variables or objects.
- **Example** :

```
let counter = 0;  
function increment() {  
  counter++; // Modifies external state  
  return counter;  
}  
console.log(increment()); // Output depends on external `counter`
```

7. ECMAScript 2015 (let,const,fat arrow):

Modern JavaScript - EcmaScript 2015

JAVASCRIPT

LET AND CONST -

Here's a comparison of **var**, **let**, and **const** in the form of a table:

Feature	var	let	const
Scope	Function scope	Block scope	Block scope
Reassignment	Can be reassigned and redeclared	Can be reassigned but not redeclared	Must be initialised and cannot be reassigned & not redeclared
Hoisting	Hoisted to the top of the function	Hoisted to the top of the block	Hoisted to the top of the block
Initialization	Can be declared without initialization	Can be declared without initialization	Must be initialised during declaration
Example	<code>var x = 10; Var x;</code>	<code>let x = 10; Let y;</code>	<code>const x = 10;</code>

- **var keyword** : it is used in JavaScript to declare variables that are function-scoped or **globally-scoped**, and can be **redeclared and reassigned** within their scope.

Example - (var redeclared)

```
var name = "mohit";  
console.log(name);
```

Output

mohit

JAVASCRIPT

```
var name = "rohit";  
console.log(name);
```

rohit

Example - (var reassigned)

Output

```
var myfavName1 = "Mohit";  
myfavName1 = "Manmohan";  
console.log(myfavName1);
```

Manmohan

- **let Keyword:** The let keyword is used to declare variables with **block scope**. Variables declared with let are mutable, meaning their values can be **reassigned but not redeclared**.

Example - (let reassigned)

Output

```
let myfavName2 = "Mohit";  
myfavName2 = "Manmohan";  
console.log(myfavName2);
```

Manmohan

Example - (let scope level)

Output

```
var age = 20;  
if (age === 20) {  
  let addition = 10;  
  console.log("valid");  
}
```

vaild
Error

```
console.log(addition);
```

- **const Keyword:** The const keyword is used to declare variables with **block scope**, but once a value is assigned to a const variable, **it cannot be reassigned. const variables are immutable and can't be redeclared**.

Const variables must be initialised during declaration.

Example - (const can't be reassigned)

Output

JAVASCRIPT

```
const myfavName = "Mohit";  
//can't be reassigned  
myfavName = "Manmohan";  
console.log(myfavName);
```

error

Example - (const scope level)

```
var age = 20;  
if (age === 20) {  
  const addition = 10;  
  console.log("valid");  
}  
console.log(addition);
```

Output

valid
Error

Here are some key points about **const** variables in JavaScript:

- **Immutable Binding:** When you declare a variable with **const**, you must initialise it with a value, and that value cannot change through reassignment.

Example:

```
const PI = 3.14;  
// PI = 3.14159; // Error: Assignment to constant variable.
```

- **Block-scoped:** Like **let**, **const** is block-scoped (scoped to the nearest curly braces { ... }).

Example :

```
if (true) {  
  const message = "Hello";  
  console.log(message); // Outputs: Hello  
}  
console.log(message); // Error: message is not defined
```

- **Object Mutability:** When **const** is used with objects or arrays, it does not prevent mutation of the object or array itself. It only prevents reassigning the variable to a different object or array.

Example :

```
const person = {  
  name: "Alice"
```

JAVASCRIPT

```
};  
person.name = "Bob"; // Allowed  
// person = { name: "Bob" }; // Error: Assignment to constant  
variable.
```

const keyword in JavaScript when used with arrays and objects.

Arrays and const:

When you declare an array using `const`, the variable holding the array reference cannot be reassigned to a different array. However, you can still modify the elements of the array.

Example :

```
const numbers = [1, 2, 3];  
numbers.push(4); // Allowed: Modifying the array  
numbers[0] = 5; // Allowed: Modifying an element
```

```
numbers = [5, 6, 7]; // Error: Assignment to constant variable
```

In the example above:

- You can add elements to the `numbers` array (`numbers.push(4);`) or modify existing elements (`numbers[0] = 5;`).
- Attempting to reassign `numbers` to a new array (`numbers = [5, 6, 7];`) will result in an error (Assignment to constant variable).

Objects and const:

Similarly, when you declare an object using `const`, the reference to the object cannot be reassigned to a different object. However, you can still modify the properties of the object.

Example :

```
const person = {  
  name: "Alice",
```

JAVASCRIPT

```
    age: 30  
};
```

```
person.age = 31; // Allowed: Modifying a property  
person.address = "123 Main St"; // Allowed: Adding a new property
```

```
person = { name: "Bob", age: 25 }; // Error: Assignment to constant  
variable
```

In this example:

- You can change the value of existing properties (`person.age = 31;`) or add new properties (`person.address = "123 Main St";`).
- Attempting to reassign a person to a completely new object (`person = { name: "Bob", age: 25 };`) will result in an error (Assignment to constant variable).

Key Points:

- **Immutability of Reference:** `const` ensures the reference (the variable itself) remains constant and cannot be reassigned to point to a different array or object.
- **Mutability of Values:** You can modify the contents (elements or properties) of the array or object declared with `const`.

TEMPLATE STRINGS -

In ECMAScript 6 (ES6), template strings, also known as template literals, provide a convenient and flexible way to create strings in JavaScript. Template strings are enclosed in **backticks** (```) rather than single or double quotes.

```
//Old ones  
let firstName = "vinod ";
```

Output

JAVASCRIPT

```
let lastName = "thapa";  
let fullName = firstName + lastName;  
console.log(fullName);
```

vinod thapa

```
//new ones  
let fName = "mohit";  
let lName = "choudhary";  
// Using template string for string interpolation  
let myName = `${fName} ${lName}`;  
console.log(myName);
```

Output

mohit choudhary

String Interpolation:

Template strings support string interpolation, // allowing you to embed expressions directly within the string. //Interpolated expressions are enclosed in \${}.

Examples :

```
let age = 29;  
// String interpolation with variable  
let message = `I am ${age} years old.`;  
console.log(message);
```

Output

I am 29 years old.

Multi-line Strings:

Template strings make it easy to create multi-line strings ,without the need for concatenation or escape characters.

```
let multiLineString = `This is a multi-line  
                        string using template literals.  
                        It's much cleaner and easier to read.`;  
console.log(multiLineString);
```

JAVASCRIPT

Expression Evaluation:

In JavaScript, expression evaluation refers to the process of interpreting and computing the result of an expression. An expression is a combination of variables, constants, operators, and function calls that produces a value. When JavaScript encounters an expression, it evaluates it to determine its resulting value.

Template expressions can include any valid JavaScript expression.

<pre>let num = 4; console.log("5 * " + num + " = " + 5 * num); console.log(`5 * \${num} = \${5 * num}`);</pre>	Output
--	---------------

Advantages:

- **Readability:** Template strings make the code more readable, especially for complex string constructions.
- **Conciseness:** They eliminate the need for explicit string concatenation and reduce the use of escape characters.
- **Expression Flexibility:** Any JavaScript expression can be embedded within `\${}`.
- **Multi-line Support:** Creating multi-line strings is more straightforward.

Default Parameters

- In ECMAScript 6 (ES6), default parameters were introduced, providing a more concise way to handle function parameter defaults.
- Default parameters allow you to specify default values for function parameters in the function declaration itself.
- If a parameter is not provided when the function is called, the default value is used.
- Default function parameters allow named parameters to be initialised with default values if no value or undefined is passed.

Example :

Output

```
let sum = function (x, y = 20) {
```

sum of 10 and 20 is 30

JAVASCRIPT

```
console.log(`sum of ${x} and ${y} is ${x + y}`);  
};  
sum(10);
```

Example :

Output

```
let sum = function (x, y = 20) {  
  console.log(`sum of ${x} and ${y} is ${x + y}`);  
};  
sum(10,100);
```

sum of 10 and 100 is 110

Default parameters are specified in the function's parameter list using the assignment (=) operator. If a parameter is not provided when calling the function or is explicitly passed as undefined, the default value is used.

Example :

```
function greet(name = 'Guest') {  
  console.log(`Hello, ${name}!`);  
}
```

```
greet();  
greet('Alice');
```

Output: Hello, Guest!
Output: Hello, Alice!

Fat Arrow Function

In JavaScript, the fat arrow function, also known as an arrow function, provides a concise way to write anonymous functions. Arrow functions were introduced in ECMAScript 6 (ES6) and offer a more compact syntax compared to traditional function expressions.

Example : Normal function declaration

```
const sum = function (a, b){  
  let result = `The sum of ${a} and ${b} is ${a + b}`;  
  console.log(result);  
};  
sum(5, 5);
```

Output

The sum of 5 and 5 is 10

JAVASCRIPT

How to convert the same in fat arrow function

Syntax

```
const functionName = (param1, param2, ...) => {  
  //Function body  
  return result; //Optional  
};
```

Example: Function declaration with fat arrow function

Output

```
const func = (a, b) => {  
  let result = `The sum of ${a} and ${b} is ${a + b}`;  
  console.log(result);  
};  
func(4, 4);
```

The sum of 4 and 4 is 8

why it is used: no need to use return and function keyword and brackets and name;

Example : Arrow function with no keywords and brackets.

```
const fun2 = (m, n) => console.log(`sum of ${m} and ${n} is ${m + n}`);  
fun2(2, 3);
```

Examples:

Output

Single parameter, implicit return

```
const square = x => x * x;  
square(2);
```

4

Multiple parameters, implicit return

```
const add = (a, b) => a + b;  
add(5, 5);
```

10

No parameters

```
const greet = () => "Hello!";  
greet();
```

Hello

NOTES

JAVASCRIPT

1: If the function body consists of a single expression, the braces {} and the return keyword can be omitted.

```
const sum = (a, b) => `The sum of ${a} and ${b} is ${a + b}`;  
console.log(sum(5, 5));
```

2: If there is only one parameter, the parentheses () around the parameter list can be omitted.

```
const square = a => `The square of ${a} is ${a * a}`;  
console.log(square(5));
```

3: If there are no parameters, use an empty set of parentheses ().

```
const greet = () => console.log("Plz LIKE SHARE & SUBSCRIBE!");  
greet();
```

Examples of Fat Arrow Function

Question 1: Calculator Function:

Using switch:

```
const calculator = (num1, num2, operator) => {  
  let result;
```

```
  switch (operator) {  
    case "+": result = num1 + num2;  
    return result;  
    break;
```

```
    case "-": result = num1 - num2;  
    return result;  
    break;
```

JAVASCRIPT

```
case "*": result = num1 * num2;
return result;
break;
```

```
case "/":
if (num2 === 0) {
return "0 is not allowed";
} else {
result = num1 / num2;
return result;
}
break;
```

```
default:
return "no operator matches";
}
};
```

```
console.log(calculator(8, 5, "+"));
console.log(calculator(8, 5, "-"));
console.log(calculator(8, 5, "/"));
console.log(calculator(8, 5, "*"));
```

Output

13
3
1.6
40

//using if else:

Question 2 : Reverse a String

Output

```
const isReverse = (str) => {
let reverse_str = "";
for (let char = str.length - 1; char >= 0; char--) {
reverse_str = reverse_str + str[char];
}
return reverse_str;
};
console.log(isReverse("vinod thapa"));
```

tihom

Question 2 : Palindrome Check:

Output

```
const isPalindrome = (str) => {
let reverse_str = "";
```

JAVASCRIPT

```
for (let char = str.length - 1; char >= 0; char--) {  
  reverse_str = reverse_str + str[char];  
}  
if (str === reverse_str) {  
  // return true;  
} else {  
  // return false;  
}  
  
//ternary operator  
return str === reverse_str ? true : false;  
};
```

```
console.log(isPalindrome("nitin"));  
console.log(isPalindrome("jamliya"));
```

True
False

8. Array in Js

- JavaScript Array is a data structure that **allows you to store and organise multiple values within a single variable**.
- It is a **versatile and dynamic object** used for storing ordered collections of values. **It can hold various data types, including numbers, strings, objects, and even other arrays**, and come with a variety of methods for performing operations on those values.
- Arrays in JavaScript are zero-indexed i.e. the first element is accessed with an index 0, the second element with an index of 1, and so forth.

Creating Arrays:

Arrays in JavaScript can be created using the **Array constructor** or with **Array literals (square brackets [])**.

JAVASCRIPT

Using Array constructor

```
let array1 = new Array("ram", "shyam", "ajay");  
console.log(array1);
```

Output

['ram', 'shyam', 'ajay']

Using array literal

```
let array2 = ["ram", "shyam", "ajay"];  
console.log(array2);
```

Output

['ram', 'shyam', 'ajay']

we can also create an empty array and object

```
let arr1 = [];  
console.log(typeof arr1);
```

Output
object

```
let arr2 = {};  
console.log(typeof arr2);
```

object

JavaScript Arrays are Versatile and Dynamic Object

Versatile

1. **Heterogeneous Elements:** JavaScript arrays can hold elements of different types. You can mix numbers, strings, booleans, objects, functions, and even other arrays within a single array.

Example: `let mixedArray = ["text", 42, true, { key: "value" }, [1, 2, 3]];`

Dynamic

1. **Resizable:** Arrays in JavaScript can grow or shrink dynamically. You don't need to define the size of the array when you create it, and you can add or remove elements at any time.

JAVASCRIPT

Example :

```
let dynamicArray = [1, 2, 3];  
  
dynamicArray.push(4); // Adds 4 to the end  
  
dynamicArray.pop(); // Removes the last element (4)
```

Output

[1,2,3,4]

2. **Dynamic Indexing:** You can access and modify elements using their index, and JavaScript will automatically adjust the array's size as needed.

Example :

```
let array = [];  
  
array[0] = "first element";  
  
array[1] = "second element";  
  
console.log(array.length);
```

Output

2

3. **Array Methods:** JavaScript provides numerous built-in methods to manipulate arrays, such as **push**, **pop**, **shift**, **unshift**, **splice**, **slice**, **map**, **filter**, **reduce**, and many others. These methods allow you to perform complex operations on arrays efficiently.

Accessing Elements:

Array elements are accessed using zero-based indices.

Example:

```
let fruits = ["apple", "banana", "orange", "grapes"];  
  
console.log(fruits[3]);
```

Output

grapes

JAVASCRIPT

Modifying Elements:

You can modify array elements by assigning new values to specific indices.

```
let fruits2 = ["apple", "banana", "orange", "grapes"];  
fruits2[1] = "mango";  
console.log(fruits2);
```

Output
['apple', 'mango', 'orange', 'grapes']

Array Traversal / Iterating Over Arrays :

Traversing or iterating over arrays in JavaScript can be done using various methods. Here are some of the most common ways to iterate over arrays:

1. For Loop

The traditional for loop is a basic way to iterate over the elements of an array.

```
let array = [1, 2, 3, 4, 5];  
  
for (let i = 0; i < array.length; i++) {  
  console.log(array[i]);  
}
```

Output
1 2 3 4 5 (vertical)

2. For...of Loop

The **for...of** loop provides a simpler syntax for iterating over the values of an iterable object.

JAVASCRIPT

for of Loop: The **for..of loop** is used to **iterate over the values of an iterable object**, such as arrays, strings, or other iterable objects.

```
let array = [1, 2, 3, 4, 5];
```

Output

```
for (let element of array) {  
  console.log(element);  
}
```

1 2 3 4 5

3. For in loop

for...in Loop: The **for...in loop** is used to iterate over the properties (**including indices**) of an object.

```
let subjects = ["dsa", "development", "javascript", "cpp", "react"];
```

Output

```
for (let item in subjects) {  
  console.log(item);  
}
```

0 1 2 3 4

```
console.log(subjects[item]); // "dsa", "development", "javascript", "cpp", "react"
```

4. ForEach Method

- The **forEach** method executes a provided function once for each array element.
- The `arr.forEach()` method calls the provided function once for each element of the array. The provided function may perform any kind of operation on the elements of the given array.

Here is a basic example and explanation of how the `forEach` method works:

Syntax :

```
array.forEach(callback(element, index, array), thisArg);
```

callback: The function to execute on each element. It takes three arguments:

- **element:** The current element being processed in the array.
- **index (optional):** The index of the current element being processed in the array.
- **array (optional):** The array `forEach` was called upon.

thisArg (optional): Value to use as `this` when executing `callback`.

JAVASCRIPT

Example : Here is a simple example using `forEach`

```
let array = [1, 2, 3, 4, 5];
```

Output

```
array.forEach(function(element) {  
  console.log(element);  
});
```

1 2 3 4 5

Example with Index :

You can also use the **index** and the **array** arguments within the **callback function**

```
let array = [1, 2, 3, 4, 5];
```

```
array.forEach(function(element, index, array) {  
  console.log('Element:', element, 'Index:', index, 'Array:', array);  
});
```

Output:

```
Element: 1 Index: 0 Array: [ 1, 2, 3, 4, 5 ]  
Element: 2 Index: 1 Array: [ 1, 2, 3, 4, 5 ]  
Element: 3 Index: 2 Array: [ 1, 2, 3, 4, 5 ]  
Element: 4 Index: 3 Array: [ 1, 2, 3, 4, 5 ]  
Element: 5 Index: 4 Array: [ 1, 2, 3, 4, 5 ]
```

Example : Arrow Function Syntax :

Using an arrow function with `forEach` makes the code more concise:

```
let array = [1, 2, 3, 4, 5];
```

Output:

```
array.forEach(ele => console.log(ele));
```

1 2 3 4 5

Practical Use Case

Here's a more practical example where we might use `forEach` to perform an operation on each element of an array, such as modifying each element or accumulating a value:

JAVASCRIPT

```
let numbers = [1, 2, 3, 4, 5];  
let sum = 0;
```

Output

```
numbers.forEach(number => {  
  
    sum += number;  
});  
console.log('Sum:', sum);
```

Sum: 15

Examples 1:

```
let subjects = ["dsa", "development", "javascript", "cpp", "react"];  
subjects.forEach((curElem, index, arr) => {  
    console.log(`${curElem} ${index}`);    });
```

Output

dsa 0

development 1

javascript 2

cpp 3

react 4

Returns Undefined

The forEach method does not return a value. It always returns undefined. It does not return any array - key difference between **forEach** and **map** function.

Example 1

Output

```
let array = [1, 2, 3, 4, 5];
```

1 2 3 4 5

```
let result = array.forEach(element => {  
    console.log(element);  
});
```

Result of forEach: **undefined**

JAVASCRIPT

```
console.log('Result of forEach:', result);
```

Example 2

```
let subjects = ["dsa", "development", "javascript", "cpp", "react"];
```

```
const myforEachArr = subjects.forEach((curElem, index, arr) => {  
return `${curElem} ${index}`;
```

```
});
```

Output

```
console.log(subject);           [ 'dsa', 'development', 'javascript', 'cpp', 'react' ]
```

```
console.log(myforEachArr);      undefined
```

5. Map Method

The **map()** method in JavaScript is used to **create a new array** by applying a function to each element of an existing array. It does not change the original array.

Here's the syntax and an example:

Syntax

```
let newArray = array.map(function(currentValue, index, arr), thisValue);
```

- **currentValue**: The current element being processed in the array.
- **index (optional)**: The index of the current element being processed in the array.
- **arr (optional)**: The array map was called upon.
- **thisValue (optional)**: A value to use as this when executing the callback function.

Example 1 :

JAVASCRIPT

Map with Arrow Function

```
let subjects = ["dsa", "development", "javascript", "cpp", "react"];

const myforMapArr = subjects.map((curElem, index, arr) => {
  return `${curElem} ${index}`;
});
console.log(subjects);
console.log(myforMapArr);
```

Output

['dsa', 'development', 'javascript', 'cpp', 'react']//**Old Array**

['dsa 0', 'development 1', 'javascript 2', 'cpp 3', 'react 4']//**New Array**

Example 2 :

Map with Normal Function

Let's say we have an array of numbers and we want to create a new array where each number is doubled.

```
let numbers = [1, 2, 3, 4];
let doubledNumbers = numbers.map(function(number) {
  return number * 2;
});
console.log(Numbers);
console.log(doubledNumbers);
```

Output: [1, 2, 3, 4]

Output: [2, 4, 6, 8]

Map with Arrow Function

```
let doubledNumbers = numbers.map(number => number * 2);
console.log(doubledNumbers);
```

Output: [2, 4, 6, 8]

Example 3 :

Output

```
let arr = [200, 100, 300, 20];

let num = arr.map((ele) => {
```

JAVASCRIPT

```
console.log(ele);           200,100,300,20
return ele * 10;
});
console.log(num);           [ 2000, 1000, 3000, 200 ]
console.log(arr);           [ 200, 100, 300, 20 ]
```

Array of Objects

(Objects inside Array)

Consider an **array of objects** representing users. We can use map to create a new array containing just the names of the users.

```
let users = [
  { name: 'Alice', age: 25 },
  { name: 'Bob', age: 30 },
  { name: 'Charlie', age: 35 }
];
```

```
let userNames = users.map(user => user.name);
```

```
console.log(userNames);
```

Output: ['Alice', 'Bob', 'Charlie']

To do Practice Time

write a program to Multiply each element with 2

```
const numbers = [1, 2, 3, 4, 5];
```

forEach - Performs an action on each element

Output
2 4 6 8 10

```
numbers.forEach((curElem) => {
  console.log(curElem * 2);
});
```

JAVASCRIPT

map - Creates a new array with transformed elements

Output

```
const doubleValue = numbers.map((curElem) => {  
  
  return curElem * 2;  
});  
console.log(doubleValue);
```

```
[ 2, 4, 6, 8, 10 ]  
//New Array
```

6.Array Methods / Properties

length Property

The length property of an array in JavaScript returns the number of elements in that array. It is a property, not a method, so you do not need to use parentheses to access it.

Example:

1. Getting the Length of an Array

Output

```
let fruits = ["apple", "banana", "cherry"];  
console.log(fruits.length);
```

3

2. Setting the Length of an Array

- If you set the length property to a value less than the current length, the array is **truncated**.
- If you set the length property to a value greater than the current length, the array is extended and the new elements are added as **undefined**.

```
let fruits = ["apple", "banana", "cherry"];  
fruits.length = 2;  
console.log(fruits);
```

["apple", "banana"]

```
fruits.length = 4;  
console.log(fruits);
```

["apple", "banana", undefined, undefined]

JAVASCRIPT

concat Method

The **concat()** method is used to merge two or more arrays. This method does not change the existing arrays but instead returns a new array.

Syntax:

array1.concat(array2, array3, ..., arrayN)

Examples:

1. Concatenating Two Arrays

Output

```
let fruits = ["apple", "banana"];  
let moreFruits = ["cherry", "date"];  
let allFruits = fruits.concat(moreFruits);  
console.log(allFruits);
```

["apple", "banana", "cherry", "date"]

2. Concatenating Multiple Arrays

Output

```
let fruits = ["apple", "banana"];  
let moreFruits = ["cherry", "date"];  
let exoticFruits = ["kiwi", "mango"];  
let allFruits = fruits.concat(moreFruits, exoticFruits);  
console.log(allFruits);
```

["apple", "banana", "cherry", "date", "kiwi", "mango"]

JAVASCRIPT

3. Concatenating Arrays and Values

Output

```
let fruits = ["apple", "banana"];  
let allFruits = fruits.concat("cherry", ["date", "kiwi"]);  
console.log(allFruits);
```

["apple", "banana", "cherry", "date", "kiwi"]

reverse method

In JavaScript, you can use the **reverse method** to reverse the order of the elements in an array. Here's an example:

```
let numbers = [1, 2, 3, 4, 5];  
numbers.reverse();  
console.log(numbers);
```

Output

[5, 4, 3, 2, 1]

This method modifies the original array in place and **returns the reversed array**.

The **reverse()** method in JavaScript **modifies the original array** in place and also **returns the same array** after reversing its elements.

If you want to reverse an array without modifying the original array, you can use the slice method to create a copy of the array first:

```
let numbers = [1, 2, 3, 4, 5];  
let reversedNumbers = numbers.slice().reverse();  
console.log(reversedNumbers);  
console.log(numbers);
```

Output: [5, 4, 3, 2, 1]

Output: [1, 2, 3, 4, 5]

In this example, `numbers.slice()` creates a shallow copy of the `numbers` array, and then `reverse` is called on this copy. This way, the original `numbers` array remains unchanged.

JAVASCRIPT

toString()

The `toString()` method in JavaScript is used to convert a value into a string representation. It is available for different data types, including numbers, arrays, objects, and booleans.

1. Using `toString()` with Numbers

```
let num = 123;  
console.log(num.toString()); // "123"
```

You can also specify a **radix** (base) to convert a number to different numeral systems:

```
let num2 = 255;  
console.log(num2.toString(2)); // "11111111" (Binary)  
console.log(num2.toString(16)); // "ff" (Hexadecimal)
```

2. Using `toString` with arrays:

Output

```
let arr = [1, 2, 3, 4];  
console.log(arr.toString());  
console.log(typeof arr.toString());  
console.log(typeof arr);
```

"1,2,3,4"
string
object

3. Using `toString()` with Objects

```
let obj = { key: "value" };  
console.log(obj.toString()); // "[object Object]"
```

In JavaScript, `split` and `join` are methods used to manipulate strings and arrays. They are often used to convert between strings and arrays by splitting a string into an array of substrings or joining an array of strings into a single string.

JAVASCRIPT

split Method() :

The split method divides a string into an array of substrings based on a specified separator.

Syntax :

string.split(separator, limit)

- **separator:** Specifies the character or regular expression to use for splitting the string. If omitted, the entire string is returned as the only element of the array.
- **limit (optional):** A non-negative integer limiting the number of splits. The resulting array will have at most limit elements.

Examples:

1. Splitting a string into an array of words:

```
const sentence = "Hello world, welcome to JavaScript";  
const words = sentence.split(" ");  
console.log(words);
```

Output : ["Hello", "world", "welcome", "to", "JavaScript"]

2. Splitting a string by a comma:

```
const csv = "apple,banana,cherry";  
const fruits = csv.split(",");  
console.log(fruits);
```

Output: ["apple", "banana", "cherry"]

3. Limiting the number of splits:

```
const data = "name:age:location";  
const limitedData = data.split(":", 2);  
console.log(limitedData);
```

Output: ["name", "age"]

The `split()` method returns a new array and does not modify the original string.

JAVASCRIPT

join Method():

The join method joins all elements of an array into a string, with a specified separator between each element.

Syntax :

array.join(separator)

separator (optional): Specifies the string to use as a separator. If omitted, a comma (,) is used by default.

Examples:

1. Joining an array of words into a sentence:

```
const words = ["Hello", "world", "welcome", "to", "JavaScript"];
const sentence = words.join(" ");
console.log(sentence);
```

Output: "Hello world welcome to JavaScript"

2. Joining an array with a different separator:

```
const fruits = ["apple", "banana", "cherry"];
const csv = fruits.join(",");
console.log(csv);
```

Output: "apple,banana,cherry"

3. Joining an array without a separator:

```
const chars = ["H", "e", "l", "l", "o"];
const word = chars.join("");
console.log(word);
```

Output: "Hello"

- `join()` does not modify the original array.
- Returns a new string with array elements joined together.
- The default separator is `,` if not specified.

JAVASCRIPT

7.Array(CRUD)

How to Create Read , Update And Delete Elements in Array(CRUD) part - 1

1: push():

Method that **adds one or more elements to the end of an array.** -
it returns the new length

Example :

```
let subjects = ["dsa", "development", "javascript", "cpp", "react"];  
console.log(subjects.push("coding")); //it returns new length  
console.log(subjects);
```

Output: **6**

```
[ 'dsa', 'development', 'javascript', 'cpp', 'react', 'coding' ]
```

2: pop():

Method that **removes the last element** from an array. -
it returns the deleted value.

Example :

```
let subjects = ["dsa", "development", "javascript", "cpp", "react"];  
console.log(subjects.pop()); //it returns the deleted value.  
console.log(subjects);
```

Output: **react**

```
[ 'dsa', 'development', 'javascript', 'cpp' ]
```

3: unshift():

Method that **adds** one or more elements to the **beginning of an array.** -
it returns the new length.

Example :

```
let subjects = ["dsa", "development", "javascript", "cpp", "react"];  
console.log(subjects.unshift("leetcode"));  
console.log(subjects);
```

Output: **6**

```
[ 'leetcode', 'dsa', 'development', 'javascript', 'cpp', 'react' ]
```

JAVASCRIPT

4: shift():

Method that **removes the first element from an array**. -
it returns the deleted value.

Example :

```
let subjects = ["dsa", "development", "javascript", "cpp", "react"];  
Const val = subjects.shift();  
console.log(val);  
console.log(subjects);
```

Output: dsa

['development', 'javascript', 'cpp']

what if, we want to add or remove anywhere in an elements - part 2

5.The splice():

The splice method in JavaScript is used to **add or remove** elements from an array. It **changes the content of an array** by removing or replacing existing elements and/or adding new elements in place. Here's how it works:

Syntax : splice(start, deleteCount, item1, item2, /* ..., */ itemN)

start: The index at which to start changing the array.

deleteCount: The number of elements to remove from the array.

item1, item2, ..., itemN: The elements to add to the array, starting from the start index. If not specified, splice() will only remove elements.

It returns the deleted value.

It modifies the original array.

Examples

1. **Removing elements :** This removes 2 elements starting from index 1.

```
let fruits = ["apple", "banana", "cherry", "date"];
```

Output

```
let result = fruits.splice(1, 2);
```

```
console.log(result);
```

['banana', 'cherry']

```
console.log(fruits);
```

['apple', 'date']

JAVASCRIPT

2. **Adding elements** : This adds "kiwi" and "mango" at index 2 without removing any elements.

```
let fruits = ["apple", "banana", "cherry", "date"];  
fruits.splice(2, 0, "kiwi", "mango");  
console.log(fruits);
```

Output

```
["apple", "banana", "kiwi", "mango", "cherry", "date"]
```

3. **Replacing elements** : This removes 2 elements starting from index 1 and adds "kiwi" and "mango" in their place.

```
let fruits = ["apple", "banana", "cherry", "date"];  
fruits.splice(1, 2, "kiwi", "mango");  
console.log(fruits);
```

Output

```
["apple", "kiwi", "mango", "date"]
```

4. **Removing all elements from a specific index** : This removes all elements from index 2 to the end of the array.

```
let fruits = ["apple", "banana", "cherry", "date"];  
fruits.splice(2);  
console.log(fruits);
```

Output

```
["apple", "banana"]
```

Important Notes

- The splice() method modifies the original array and returns an array of the removed elements, if any.
- If deleteCount is 0 or negative, no elements are removed.
- If start is greater than the length of the array, it will start at the end of the array.
- If deleteCount is omitted, all elements from start to the end of the array are removed.

JAVASCRIPT

Todo Challenge time

Example :

```
const months = ["Jan", "march", "April", "June", "July"];
```

Output

Que 1: Add Dec at the end of an array?

```
//months.push("Dec");
```

```
months.splice(months.length, 0, "Dec");
```

```
console.log(months);           [ 'Jan', 'march', 'April', 'June', 'July', 'Dec' ]
```

Que 2: What is the return value of the splice method

```
console.log(months.splice());  []//deleted Values
```

Que 3: Update march to March (update)?

```
// months[1] = "March";
```

```
const indextoupdate = months.indexOf("march");
```

```
months.splice(indextoupdate, 1, "March");
```

```
console.log(months);           [ 'Jan', 'March', 'April', 'June', 'July', 'Dec' ]
```

Que 4: Delete June from an array?

```
//months.splice(3, 1);
```

```
const indexToDelete = months.indexOf("June");
```

```
months.splice(indexToDelete, 1);
```

```
console.log(months);           [ 'Jan', 'March', 'April', 'July', 'Dec' ]
```

JAVASCRIPT

6. The slice()

The slice() method in JavaScript is used to **create a shallow copy of a portion of an array into a new array object**. This method does not modify the original array.

Syntax : `array.slice(start, end)`

start (optional): The beginning index of the portion of the array to be extracted. If omitted, it defaults to 0.

end (optional): The end index (**exclusive**) of the portion of the array to be extracted. If omitted, it extracts through the end of the array.

Examples

1. **Basic Usage :** In this example, the slice() method creates a new array containing elements from index 1 to 3 (excluding 3).

<code>let fruits = ["apple", "banana", "cherry", "date"];</code>	Output
<code>let citrus = fruits.slice(1, 3);</code>	
<code>console.log(citrus);</code>	<code>["banana", "cherry"]</code>
<code>console.log(fruits);</code>	<code>["apple", "banana", "cherry", "date"]</code>

2. **Omitting end Parameter :** Here, omitting the end parameter means the slice includes all elements from index 1 to the end of the array.

<code>let fruits = ["apple", "banana", "cherry", "date"];</code>	
<code>let citrus = fruits.slice(1);</code>	Output
<code>console.log(citrus);</code>	<code>["banana", "cherry", "date"]</code>

3. **Omitting Both Parameters :** Omitting both start and end parameters creates a copy of the entire array.

JAVASCRIPT

```
let fruits = ["apple", "banana", "cherry", "date"];
```

```
let citrus = fruits.slice();
```

Output

```
console.log(citrus);
```

```
["apple", "banana", "cherry", "date"]
```

- 4. Using Negative Indices :** Negative indices can be used to specify offsets from the end of the array. In this example, -3 starts the slice from the **third-to-last** element, and -1 ends the slice at the **second-to-last** element.

```
let fruits = ["apple", "banana", "cherry", "date"];
```

Output

```
let citrus = fruits.slice(-3, -1);
```

```
console.log(citrus);
```

```
["banana", "cherry"]
```

5. Zero as end Parameter

If end is 0, the slice will start from the start index and return an empty array since end is exclusive and 0 is before the start index.

```
let fruits = ["apple", "banana", "cherry", "date"];
```

Output

```
let citrus = fruits.slice(1, 0);
```

```
console.log(citrus);
```

```
[]
```

Important Notes

- The slice() method does not modify the original array; it returns a new array.
- If start is greater than or equal to the length of the array, slice() returns an empty array.
- If end is omitted, slice() extracts through the end of the array.
- If either start or end is negative, it is treated as $\text{array.length} + \text{start}$ or $\text{array.length} + \text{end}$ respectively.

JAVASCRIPT

differences between the slice() and splice() methods in JavaScript:

Feature	slice()	splice()
Purpose	To create a shallow copy of a portion of an array	To add, remove, or replace elements in an array
Syntax	array.slice(start, end)	array.splice(start, deleteCount, item1, item2, ..., itemN)
Modifies Original Array	No	Yes
Return Value	A new array with the extracted elements	An array containing the deleted elements
Parameters		
- start	Yes (optional, default is 0)	Yes
- end	Yes (optional, default is array's length)	No
- deleteCount	N/A	Yes (optional, default is to remove all from start)
- Items to add	N/A	Modifying the array by adding, removing, or replacing elements
Use Case	Copying part of an array without modifying the original	Modifying the array by adding, removing, or replacing elements
Example: Removing Elements	let result = array.slice(1, 3);	let result = array.splice(1, 2);
Example: Adding Elements	N/A	array.splice(2, 0, "new1", "new2");
Example: Replacing Elements	N/A	array.splice(1, 2, "new1", "new2");
Negative Indices	Supported	Not Supported

JAVASCRIPT

8. Searching and Filter() in an Array

Searching in an Array

There are several methods in JavaScript to search for elements within an array. Each method serves different purposes and can be used depending on the requirements.

1. indexOf()

This method returns the first index at which a given element can be found in the array, or -1 if it is not present.

syntax :

```
indexOf(searchElement);  
indexOf(searchElement, fromIndex);
```

Example 1:

```
let arr = [1, 2, 3, 4, 5];  
console.log(arr.indexOf(3));  
console.log(arr.indexOf(6));
```

Output

2
-1

Example 2:

```
const numbers1 = [1, 2, 3, 4, 5, 6, 7, 8, 9];  
console.log(numbers1.indexOf(4));  
console.log(numbers1.indexOf(4, 4)); // it returns -1 because we starts  
searching from index 4
```

Output

3

2. lastIndexOf()

This method returns the last index at which a given element can be found in the array, or -1 if it is not present.

Example 1:

```
let arr = [1, 2, 3, 4, 3, 5];  
console.log(arr.lastIndexOf(3));  
console.log(arr.lastIndexOf(6));
```

Output

4
-1

Example 2:

```
const numbers2 = [1, 2, 3, 4, 5, 6, 7, 6, 9];
```

Output

JAVASCRIPT

```
const result = numbers2.indexOf(6);  
console.log(result);
```

5

```
const result2 = numbers2.lastIndexOf(6);  
// it comes backward in array for searching  
console.log(result2);
```

7

```
const result2 = numbers2.lastIndexOf(6,6);  
// it comes backward in array for searching  
console.log(result2);
```

5

3. includes()

This method determines whether an array includes a certain value among its entries, returning true or false as appropriate.

Syntax : includes(searchElement)
includes(searchElement, fromIndex)

Example 1:

Output

```
let arr = [1, 2, 3, 4, 5];  
console.log(arr.includes(3));  
console.log(arr.includes(6));
```

true
false

4. some()

This method tests whether **at least one element in the array passes the test** implemented by the provided function. It returns a Boolean value.

Example 1:

Output

```
let arr = [1, 2, 3, 4, 5];  
let hasEvenNumber = arr.some(element => element % 2 === 0);  
console.log(hasEvenNumber);
```

true

JAVASCRIPT

5. every()

This method tests whether **all elements in the array pass the test** implemented by the provided function. It returns a Boolean value.

Example 1:

```
let arr = [1, 2, 3, 4, 5];
```

```
let allLessThanSix = arr.every(element => element < 6);
```

```
console.log(allLessThanSix);
```

Output

true

6. find()

This method returns the value of the **first element in the array** that satisfies the provided testing function. Otherwise, it **returns undefined**.

Example :

```
let arr = [1, 2, 3, 4, 5];
```

```
let found = arr.find(element => element > 3);
```

```
console.log(found);
```

Output

4

7. findIndex()

This method returns the **index of the first element in the array** that satisfies the provided testing function. Otherwise, it returns -1.

Example :

```
let arr = [1, 2, 3, 4, 5];
```

```
let foundIndex = arr.findIndex(element => element > 3);
```

```
console.log(foundIndex);
```

Output

3

8. findLastIndex()

This method returns the **index of the last element in the array** that satisfies the provided testing function. Otherwise, it returns -1.

Example :

```
let arr = [1, 2, 3, 4, 3, 5];
```

```
let foundIndex = arr.findLastIndex(element => element > 3);
```

```
console.log(foundIndex);
```

Output

5

JAVASCRIPT

9.filter()

The filter method in JavaScript **creates a new array** with all elements that pass the test implemented by the provided function. It doesn't change the original array.

Syntax:

```
filter(callbackFn)
```

```
filter(callbackFn, thisArg)
```

```
array.filter(callback(element[, index[, array]]), thisArg)
```

callback: A function that is called for every element of the array. It should return a value that coerces to true to keep the element, or false otherwise.

- **element:** The current element being processed in the array.
- **index (optional):** The index of the current element being processed in the array.
- **array (optional):** The array filter was called upon.

thisArg (optional): Value to use as this when executing callback.

Simple Example:

Output

```
const numbers = [1, 2, 3, 4, 5, 4, 6, 7, 8, 6, 9];
```

```
const result = numbers.filter((currEle) => {  
  return currEle > 5;  
});
```

```
console.log(result); //copy arr
```

```
[ 6, 7, 8, 6, 9 ]
```

```
console.log(numbers);//original arr
```

```
[1, 2, 3, 4, 5, 4, 6, 7, 8, 6, 9]
```

JAVASCRIPT

Examples 1 : Filtering an array of numbers:

Output

```
let numbers = [1, 2, 3, 4, 5, 6];  
  
let evenNumbers = numbers.filter(function(number) {  
  return number % 2 === 0;  
});  
  
console.log(evenNumbers);
```

[2, 4, 6]

Examples 2 : Using arrow functions:

Output

```
let numbers = [1, 2, 3, 4, 5, 6];  
  
let oddNumbers = numbers.filter(number => number % 2 !== 0);  
  
console.log(oddNumbers);
```

[1, 3, 5]

Example 3 : Filtering objects in an array:

```
let people = [  
  { name: "Alice", age: 25 },  
  { name: "Bob", age: 30 },  
  { name: "Charlie", age: 35 }  
];  
  
let adults = people.filter(person => person.age >= 30);  
  
console.log(adults);
```

Output

```
[{ name: "Bob", age: 30 }, { name: "Charlie", age: 35 }]
```

JAVASCRIPT

Example 4: Filtering with index:

```
let numbers = [10, 20, 30, 40, 50];
```

```
let filteredNumbers = numbers.filter((number, index) => index % 2 === 0);
```

```
console.log(filteredNumbers);
```

Output [10, 30, 50]

Example 5 :Using thisArg:

```
let numbers = [1, 2, 3, 4, 5, 6];
```

```
let obj = { threshold: 3 };
```

```
let greaterThanThreshold = numbers.filter(function(number) {
```

```
    return number > this.threshold;
```

```
}, obj);
```

```
console.log(greaterThanThreshold);
```

Output [4, 5, 6]

The filter method is very powerful for creating subsets of arrays based on specific criteria.

Questions: Filter unique values

```
const numbers = [1, 2, 3, 4, 6, 5, 6, 7, 8, 9];
```

Output

```
let uniqueNum = numbers.filter((curEle, index, arr) => {
```

```
//console.log(index);
```

[1, 2, 3, 4, 6, 5, 7, 8, 9]

```
//console.log(arr.indexOf(curElem));
```

```
return arr.indexOf(curEle) === index;
```

```
});
```

//using set method

Output

```
console.log([...new Set(numbers)])
```

[1, 2, 3, 4, 6, 5, 7, 8, 9]

JAVASCRIPT

9. Sort and Compare an Array

Sorting an Array

To sort an array in JavaScript, you can use the sort method. By default, sort converts elements to **strings** and **sorts them in ascending order**, which may not always be desirable, especially for numbers.

The `sort()` method in JavaScript is used to sort an array **in place**, meaning it modifies the original array. By default, it sorts elements as **strings** in **ascending order**.

Sorting strings

Alphabetical Order:

```
const fruits = ["Banana", "Apple", "Orange", "Mango"];
```

```
fruits.sort();
```

```
console.log(fruits);
```

Output : ['Apple', 'Banana', 'Mango', 'Orange']

Reverse Alphabetical Order:

```
let fruits = ["banana", "apple", "cherry"];
```

```
fruits.sort((a, b) => b.localeCompare(a));
```

```
console.log(fruits);
```

Output: ["cherry", "banana", "apple"]

Sorting Numbers

sort function does not work with the numbers

Example:

```
const arr_num = [23, 4, 2, 45, 6, 4, 2];
```

```
arr_num.sort();
```

```
console.log(arr_num); //wrong sorting
```

Output

[2, 2, 23, 4, 4, 45, 6]

JAVASCRIPT

For sorting numbers in ascending or descending order, you need to provide a comparison function.

compare callback function

//if (a>b) return 1 => switch the order

//if (b>a) return -1 => keep the order

Ascending Order:

```
const arr_num = [23, 4, 2, 45, 6, 4, 2];
```

```
arr_num.sort((a, b) => {
```

```
  if (a > b) return 1;
```

```
  if (b > a) return -1;
```

```
});
```

```
console.log(arr_num);
```

Output

[2, 2, 4, 4, 6, 23, 45]

//Shorthand

```
let numbers = [4, 2, 5, 1, 3];
```

```
numbers.sort((a, b) => a - b);
```

```
console.log(numbers);
```

Output: [1, 2, 3, 4, 5]

Descending Order:

```
const arr_num = [23, 4, 2, 45, 6, 4, 2];
```

```
arr_num.sort((a, b) => {
```

```
  if (a > b) return -1;
```

```
  if (b > a) return 1;
```

```
});
```

```
console.log(arr_num);
```

Output

[45, 23, 6, 4, 4, 2, 2]

JAVASCRIPT

//Shorthand

```
let numbers = [4, 2, 5, 1, 3];
```

```
numbers.sort((a, b) => b - a);
```

```
console.log(numbers);
```

Output: [5, 4, 3, 2, 1]

Key Points

- ✓ `sort()` modifies the original array (it does not return a new one).
- ✓ For numbers, always use a `compareFunction` to avoid incorrect sorting.
- ✓ For objects, sort based on a key (e.g., `a.age - b.age`).
- ✓ Use `localeCompare()` for case-insensitive string sorting.

Comparing Arrays

To compare two arrays for equality (i.e., to check if they have the same elements in the same order), you can use the following approach:

```
function arraysEqual(arr1, arr2) {  
  if (arr1.length !== arr2.length) {  
    return false;  
  }  
  for (let i = 0; i < arr1.length; i++) {  
    if (arr1[i] !== arr2[i]) {  
      return false;  
    }  
  }  
  return true;  
}  
let array1 = [1, 2, 3];  
let array2 = [1, 2, 3];  
let array3 = [1, 2, 4];
```

```
console.log(arraysEqual(array1, array2));  
console.log(arraysEqual(array1, array3)); //
```

Output: true

Output: false

JAVASCRIPT

10.Reduce method

The reduce method in JavaScript is used to accumulate or reduce an array to a single value. It iterates over the elements of an array and applies a callback function to each element, updating an accumulator value with the result.

The reduce method takes a **callback function** as its first argument and an optional **initial value** for the accumulator as the second argument.

syntax

```
array.reduce(function callback(accumulator, currentValue, index, array) {  
  // Your logic here  
  // Return the updated accumulator value  
  // }, initialValue);
```

callback: A function that is called once for each element in the array, taking four arguments:

- **accumulator:** The accumulated value previously returned in the last invocation of the callback, or initialValue, if supplied.(**The accumulated result of the previous iterations.**)
- **currentValue:** The current element being processed in the array.
- **index (optional):** The index of the current element being processed in the array.
- **array (optional):** The array reduce was called upon.

initialValue (optional): A value to use as the first argument to the first call of the callback. If no initial value is supplied, the first element in the array will be used as the initial accumulator value, and the callback will start from the second element.

Examples:

1. Sum of all elements in an array:

```
const numbers = [1, 2, 3, 4, 5];  
const sum = numbers.reduce((accumulator, currentValue) =>  
  accumulator + currentValue, 0);  
console.log(sum);
```

Output: 15

JAVASCRIPT

2. Flattening an array of arrays:

```
const arrays = [[1, 2], [3, 4], [5, 6]];
const flattened = arrays.reduce((accumulator, currentValue) =>
  accumulator.concat(currentValue), []);
console.log(flattened);
```

Output: [1, 2, 3, 4, 5, 6]

3. Counting instances of values in an object:

```
const names = ['Alice', 'Bob', 'Tiff', 'Bruce', 'Alice'];
const nameCount = names.reduce((acc, name) => {
  acc[name] = (acc[name] || 0) + 1;
  return acc;
}, {});
console.log(nameCount);
```

Output: { Alice: 2, Bob: 1, Tiff: 1, Bruce: 1 }

4. Grouping objects by a property:

```
const people = [
  { name: 'Alice', age: 21 },
  { name: 'Bob', age: 22 },
  { name: 'Alice', age: 23 }
];
const groupedByName = people.reduce((acc, person) => {
  const { name } = person;
  if (!acc[name]) {
    acc[name] = [];
  }
  acc[name].push(person);
  return acc;
}, {});
console.log(groupedByName);
```

Output:

```
{
  Alice: [{ name: 'Alice', age: 21 }, { name: 'Alice', age: 23 }],
  Bob: [{ name: 'Bob', age: 22 }]
}
```

JAVASCRIPT

QUESTIONS :

Question 1 : using the map method, write a function that takes array of strings and return a new array where each string is capitalised;

//toUpperCase methods

```
const mystr = ["apple", "banana", "cherry", "date"];
```

Output

```
const outcome = mystr.map((curEle) => {
```

```
return curEle.toUpperCase();
```

```
});console.log(outcome);
```

['APPLE', 'BANANA', 'CHERRY', 'DATE']

//toLowerCase methods

```
const mystr1 = ["MOHIT", "MANMOHAN", "MANMOHIT", "MIT"];
```

```
const outcome1 = mystr1.map((curEle) => {
```

```
return curEle.toLowerCase();
```

```
});
```

```
console.log(outcome1);
```

['mohit', 'manmohan', 'manmohit', 'mit']

Question 2 : using the map method, write a function that takes a array of numbers and return a new array where each number is squared , but only if it's an even number

```
const names = ["ram", "shyam", "sumit", "mohit"];
```

```
const prefixName = names.map((currEle) => {
```

```
return (currEle = `Mr. ${currEle}`);
```

```
});
```

```
console.log(prefixName);
```

Output : ['Mr. ram', 'Mr. shyam', 'Mr. sumit', 'Mr. mohit']

//shorthand

JAVASCRIPT

```
const prefixName1 = names.map((currEle) => `Mr. ${currEle}`);  
console.log(prefixName1);
```

Output : ['Mr. ram', 'Mr. shyam', 'Mr. sumit', 'Mr. mohit']

Question : 3 write a javascript function that calculates the total price of items in a shopping cart. The function should take an array of item prices as input and return the total price.

```
const productPrices = [100, 200, 230, 2304, 902, 234, 897];  
  
const totalPrice = productPrices.reduce((accum, curEle) => {  
    return accum + curEle;  
}, 0);  
  
console.log(totalPrice);
```

Output : 4867

8.2 Multi Dimensional Array :

A multi-dimensional array in JavaScript is essentially an **array of arrays**.

Creating a Multidimensional Array

A common example is a two-dimensional array, which can be visualised as a grid or a table.

Creating a 2D array

```
let array2D = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
];
```

Accessing Elements

```
console.log(array2D[0][0]);
```

Output: 1

JAVASCRIPT

```
console.log(array2D[1][2]);
```

Output: 6

Modifying Elements

```
array2D[2][1] = 10;  
console.log(array2D[2][1]);
```

Output: 10

Iterating Through a Multidimensional Array

You can use nested loops to iterate through a multidimensional array:

```
for (let i = 0; i < array2D.length; i++) {  
  for (let j = 0; j < array2D[i].length; j++) {  
    console.log(array2D[i][j]);  
  }  
}
```

Output

1,2,3,4,5,6,7,8,9

Example :

```
let arr = [  
  ["Johnethin", 30], //0  
  ["Mike", 20],      //1  
  ["el", 18],        //2  
  ["Nancy", 26],     //3  
];
```

//To find Length of arr

```
console.log(arr.length);
```

Output

4

//To Print arr

```
console.log(arr);
```

[['Johnethin', 30], ['Mike', 20],
 ['el', 18], ['Nancy', 26]]

//To Print single element from arr

```
console.log(arr[0]);
```

['Johnethin', 30]

//To access single value from arr

```
console.log(arr[0][0]);
```

Johnethin

//Iterate Using nested for loop

```
for (let i = 0; i < arr.length; i++) {  
  for (let j = 0; j < arr[i].length; j++) {  
    console.log(arr[i][j]);  
  }  
}
```

JAVASCRIPT

//Using for each

```
arr.forEach((ele) => {  
  ele.forEach((val) => {  
    console.log(val);  
  });  
});
```

//Using for of

```
for (let ar of arr) {  
  for (let v of ar) {  
    console.log(v);  
  }  
}
```

Output : ALL LOOP OUTCOME

```
Johnethin  
30  
Mike  
20  
el  
18  
Nancy  
26
```

Array Destructuring

Array destructuring is a convenient way of extracting values from arrays and assigning them to variables. It is a syntactic feature introduced in ES6 (ECMAScript 2015).

Example :

Traditional way of accessing array elements

```
const arr = [1, 2, 3];  
const first = arr[0];  
const second = arr[1];  
const third = arr[2];  
console.log(first, second, third);
```

Output: 1 2 3

Using array destructuring

```
const [a, b, c] = arr;
```

JAVASCRIPT

```
console.log(a, b, c);
```

Output: 1 2 3

Key Features of Array Destructuring

1. **Skipping Elements** : You can skip elements in the array by leaving gaps between commas.

```
const arr = [1, 2, 3, 4];
```

```
const [first, , third] = arr;
```

```
console.log(first, third);
```

Output: 1 3

2. **Default Values** : If the array doesn't have enough elements, you can assign default values.

```
const arr = [1, 2];
```

```
const [a, b, c = 3] = arr; console.log(a, b, c);
```

Output: 1 2 3

3. **Swapping Variables** : Array destructuring provides a concise way to swap variable values.

```
let x = 1;
```

```
let y = 2;
```

```
[x, y] = [y, x];
```

```
console.log(x, y);
```

Output: 2 1

4. **Nested Destructuring** : You can destructure nested arrays in a similar manner.

```
const nestedArr = [1, [2, 3], 4];
```

```
const [a, [b, c], d] = nestedArr;
```

```
console.log(a, b, c, d);
```

Output: 1 2 3 4

5. **Rest Pattern** : You can use the rest pattern (...) to collect the remaining elements into a new array.

JAVASCRIPT

```
const arr = [1, 2, 3, 4, 5];
```

```
const [first, second, ...rest] = arr;
```

```
console.log(first);
```

Output: 1

```
console.log(second);
```

Output: 2

```
console.log(rest);
```

Output: [3, 4, 5]

Array destructuring in JavaScript makes it easier to work with arrays by providing a clear and concise syntax to extract and assign values.

- Type of arrays is OBJECT
- In javascript internally array is an object

9. Copy and Spread :

1. Using Assignment Operator :

Output

```
let arr1 = ["mohit", 101, true];  
let arr2 = arr1;
```

```
console.log(arr1);  
console.log(arr2);
```

['mohit', 101, true]
['mohit', 101, true]

```
arr1.push("copy Arr1");  
//this method push ele on both arrays  
console.log(arr1);  
console.log(arr2);
```

['mohit', 101, true, 'copy Arr1']
['mohit', 101, true, 'copy Arr1']

```
arr2.push("copy Arr2");  
//this method push ele on both arrays  
console.log(arr1);  
console.log(arr2);
```

['mohit', 101, true, 'copy Arr1', 'copy Arr2']
['mohit', 101, true, 'copy Arr1', 'copy Arr2']

In JavaScript, copying and spreading elements of arrays or properties of objects is commonly done using the **spread operator (...)** or methods like **Object.assign()** for objects. Here's a detailed explanation and examples:

JAVASCRIPT

Spread Operator (...)

The spread operator allows you to spread elements of an array or properties of an object into another array or object.

Example: Copying Arrays

```
const originalArray = [1, 2, 3];  
const copiedArray = [...originalArray];
```

```
console.log(copiedArray);
```

Output: [1, 2, 3]

This creates a shallow copy of the original array. Modifying the copied array won't affect the original array.

Example :Copying Objects

```
const originalObject = { a: 1, b: 2 };  
const copiedObject = { ...originalObject };
```

```
console.log(copiedObject);
```

Output: { a: 1, b: 2 }

JAVASCRIPT

10. Shallow & Deep Copy :

Shallow Copy

- A shallow copy means that the copy of an **object or array** is a new instance, but the elements or properties inside the object or array are still **references to the same memory locations** as those in the original object or array.
- A shallow copy means that only the first level of the object is copied, and the **nested objects or arrays are still references to the original ones**. You can create a shallow copy using **Object.assign()** or the **spread operator (...)**.
- This contrasts with a deep copy, where all elements or properties are completely independent copies, including nested objects or arrays.
- **For top-level elements (primitives like numbers, strings, booleans, etc.) → They have different memory.**
- **✗ For nested objects/arrays → They share the same memory (reference).**

Example : Shallow Copy of an Array

```
const originalArray = [1, 2, { a: 3 }];  
const shallowCopiedArray = [...originalArray];  
  
shallowCopiedArray[2].a = 10;           //change reflect on both arrays  
console.log(originalArray);              Output: [1, 2, { a: 10 }]  
console.log(shallowCopiedArray);         Output: [1, 2, { a: 10 }]
```

In this example, both `originalArray` and `shallowCopiedArray` reference the same object at index 2. Changing the object in `shallowCopiedArray` also changes it in `originalArray`.

Example : Shallow Copy of an Object

```
const originalObject = { a: 1, b: { c: 2 } };  
const shallowCopiedObject = { ...originalObject };  
shallowCopiedObject.b.c = 10;           //change reflect on both arrays  
console.log(originalObject);             Output: { a: 1, b: { c: 10 } }  
console.log(shallowCopiedObject);        Output: { a: 1, b: { c: 10 } }
```

JAVASCRIPT

Similarly, in this example, both `originalObject` and `shallowCopiedObject` reference the same nested object in property `b`. Modifying `shallowCopiedObject.b.c` affects `originalObject.b.c`.

Shallow Copy Methods

- **Spread Operator (...)**: Creates a shallow copy of an array or object.
- **Object.assign()**: Creates a shallow copy of an object.

Limitations of Shallow Copy

A shallow copy is suitable for simple objects and arrays that don't contain other objects or arrays. If you have nested structures and you want a completely independent copy, you need a deep copy.

Deep Copy

A deep copy means that all levels of nested objects or arrays are copied, and the copy does not share any references with the original object or array.

In **deep copy**, both the **top-level** and **nested objects/arrays** get completely **new memory locations**. This means modifying one object **won't affect the other**.

Deep Copy Example with objects

```
const originalObject = { a: 1, b: { c: 2 } };  
const deepCopiedObject = JSON.parse(JSON.stringify(originalObject));
```

```
deepCopiedObject.b.c = 10;
```

```
console.log(originalObject);      Output: { a: 1, b: { c: 2 } }  
console.log(deepCopiedObject);    Output: { a: 1, b: { c: 10 } }
```

In this example, `deepCopiedObject` is a completely independent copy of `originalObject`, including the nested object in property `b`.

For nested objects or arrays, both the spread operator and `Object.assign()` create a shallow copy. To create a deep copy, you can use methods like `JSON.parse(JSON.stringify(obj))` or libraries like `Lodash`.

JAVASCRIPT

Deep Copy Example with Arrays

Creating a deep copy of an array ensures that not only the array itself is copied, but also any nested objects or arrays within it.

Using `JSON.parse(JSON.stringify())`

- The most straightforward way to create a deep copy of an array in JavaScript is by using `JSON.parse(JSON.stringify())`.
- This method works well for arrays containing serializable data (i.e., data that can be represented in JSON format).

Example:

```
let originalArray = [1, 2, {a: 1, b: 2}, [3, 4]];
```

```
let deepCopiedArray = JSON.parse(JSON.stringify(originalArray));
```

```
deepCopiedArray[2].a = 10;
```

```
deepCopiedArray[3].push(5);
```

```
console.log("Original Array:", originalArray);
```

Outputs: [1, 2, {a: 1, b: 2}, [3, 4]]

```
console.log("Deep Copied Array:", deepCopiedArray);
```

Outputs: [1, 2, {a: 10, b: 2}, [3, 4, 5]]

JAVASCRIPT

11. Objects in js

What is Object

Objects are a fundamental part of JavaScript, providing a way to group related data and functions together. In JavaScript, an **object** is a collection of **key-value pairs**. Each key (also called a **property or attribute**) is a **string**, and each value can be any data type, including another object. Objects are used to store and manage data in a structured way.

Key Concepts of JavaScript Objects:

1. **Properties:** Values associated with an object. These are defined as key-value pairs.

```
let car = {  
  make: 'Toyota',  
  model: 'Camry',  
  year: 2020  
};
```

2. **Methods:** Functions that are properties of an object.

```
let person = {  
  firstName: 'John',  
  lastName: 'Doe',  
  fullName: function() {  
    return this.firstName + ' ' + this.lastName;  
  }  
};
```

3. **Creating an Object:** There are several ways to create objects in JavaScript:

- **Object Literals:**

```
let obj = {  
  property1: 'value1',  
  property2: 'value2'  
};
```

JAVASCRIPT

- **Using the new Object() Syntax :**

```
let obj = new Object();  
obj.property1 = 'value1';  
obj.property2 = 'value2';
```

- **Using a Constructor Function:**

```
function Person(firstName, lastName) {  
    this.firstName = firstName;  
    this.lastName = lastName;  
}  
  
let person1 = new Person('John', 'Doe');  
console.log(person1);
```

Output : Person { firstName: 'John', lastName: 'Doe' }

4. Accessing Object Properties:

- **Dot Notation:**

```
console.log(car.make);
```

Output: Toyota

- **Bracket Notation:**

```
console.log(car['model']);
```

Output: Camry

5. Modifying Properties:

```
car.year = 2021;  
car['make'] = 'Honda';
```

6. Adding New Properties:

```
car.color = 'red';
```

7. Deleting Properties:

```
delete car.model;
```

JAVASCRIPT

Example of a JavaScript Object : The most common one is using object literals:

```
let StrangerThings = {  
  title: "Stranger Things",  
  creators: "The Duffer Brothers",  
  type: "American science fiction drama.",  
  publishYear: 2016,  
  getSummary: function () {  
    return `${this.title} by ${this.creators}, published in ${this.publishYear}, It is a  
type of ${this.type}`;  
  },  
};
```

```
console.log(StrangerThings.getSummary());
```

Output :

Stranger Things by The Duffer Brothers, published in 2016, It is a type of American science fiction drama.

JAVASCRIPT

Objects Useful Methods & Concepts :

1. Nested Objects : Objects can contain other objects as properties, creating a nested structure.

```
let student = {  
  name: 'Alice',  
  age: 20,  
  address: {  
    street: '123 Main St',  
    city: 'Wonderland'  
  }  
};
```

```
console.log(student.address.city);
```

Output: Wonderland

2. Object Methods : Methods are functions that belong to an object. They can perform actions using the object's properties.

```
let calculator = {  
  add: function(a, b) {  
    return a + b;  
  },  
  subtract: function(a, b) {  
    return a - b;  
  }  
};
```

```
console.log(calculator.add(5, 3));
```

Output: 8

```
console.log(calculator.subtract(5, 3));
```

Output: 2

//Method Shorthand :

```
let calculator = {  
  add(a, b) {  
    return a + b;  
  },  
  subtract(a, b) {  
    return a - b;  
  }  
};
```

```
console.log(calculator.add(5, 3));
```

Output: 8

```
console.log(calculator.subtract(5, 3));
```

Output: 2

JAVASCRIPT

3. this Keyword : The this keyword refers to the current object. It is often used inside methods to access the object's properties.

```
let dog = {  
  name: 'Buddy',  
  breed: 'Golden Retriever',  
  description: function() {  
    return `${this.name} is a ${this.breed}`;  
  }  
};
```

```
console.log(dog.description());
```

Buddy is a Golden Retriever

4. Object Constructors : Constructor functions allow you to create multiple instances of similar objects.

```
function Car(make, model, year) {  
  this.make = make;  
  this.model = model;  
  this.year = year;  
}  
  
let car1 = new Car('Toyota', 'Corolla', 2021);  
let car2 = new Car('Honda', 'Civic', 2022);
```

```
console.log(car1.make);  
console.log(car2.model);
```

Output: Toyota

Output: Civic

6. Object.keys() and Object.values() : These methods allow you to get an array of a given object's keys or values.

```
let user = {  
  name: 'Alice',  
  age: 25,  
  city: 'Wonderland'  
};
```

```
console.log(Object.keys(user));  
console.log(Object.values(user));
```

Output

['name', 'age', 'city']

['Alice', 25, 'Wonderland']

JAVASCRIPT

7. Object.assign() : Another way to copy objects is using Object.assign(). This method copies all enumerable properties from one or more source objects to a target object.

Example: Copying Objects

```
const originalObject = { a: 1, b: 2 };  
const copiedObject = Object.assign({}, originalObject);
```

```
console.log(copiedObject);
```

Output: { a: 1, b: 2 }

8. Object.entries() : This method returns an array of a given object's own enumerable string-keyed property [key, value] pairs.

```
let book = {  
  title: '1984',  
  author: 'George Orwell',  
  year: 1949  
};  
console.log(Object.entries(book));
```

Output: [['title', '1984'], ['author', 'George Orwell'], ['year', 1949]]

9. Object Destructuring : Object destructuring allows you to unpack values from objects into distinct variables.

```
let person = {  
  name: 'Alice',  
  age: 25,  
  city: 'Wonderland'  
};
```

```
let { name, age, city } = person;
```

```
console.log(name);  
console.log(age);  
console.log(city);
```

Output

Alice
25
Wonderland

You can also assign new variable names during destructuring.

```
let { name: personName, age: personAge } = person;  
console.log(personName);  
console.log(personAge);
```

Alice
25

JAVASCRIPT

10. Spread Operator : The spread operator (...) allows you to create a shallow copy of an object or merge multiple objects.

```
let obj1 = { a: 1, b: 2 };  
let obj2 = { b: 3, c: 4 };  
let mergedObj = { ...obj1, ...obj2 };
```

```
console.log(mergedObj);
```

Output: { a: 1, b: 3, c: 4 }

We can add dynamic keys in an object:

```
let idType = "studentId";
```

```
let student = {  
  [idType]: "A123456",           //Dynamic key based on idType  
  sName: "Vinod",  
  sAge: 29,  
  isStudent: true,  
  greet: function () {  
    console.log( `Hey, my ${idType} is ${student[idType]} and my name is  
    ${student.sName}` ); },  
};student.greet();
```

Output : Hey, my studentId is A123456 and my name is Vinod.

useCase: when we want to get the user name and value in react

11. Object.freeze() : Freezes an object, preventing new properties from being added and existing properties from being **removed or modified**.

```
let obj = { prop: 42 };  
Object.freeze(obj);  
obj.prop = 33; //This will not work  
console.log(obj.prop);
```

Output: 42

12. Object.seal(): Seals an object, preventing new properties from being added and marking all existing properties as **non-configurable**.

```
let obj2 = { prop: 42 };  
Object.seal(obj2);  
obj2.newProp = 33; //This will not work  
console.log(obj2.newProp);
```

Output: undefined

JAVASCRIPT

Object of Arrays

(Arrays inside object) :example where objects contain arrays as properties.

```
let objectofArray= {  
  arr1: [1, 2, 3, 4],  
  arr2: ["mohit", "rohit"],  
};
```

Output

1

```
{ arr1: [ 1, 2, 3, 4 ], arr2: [ 'mohit', 'rohit' ] }
```

```
console.log(objectofArray.arr1[0]);  
console.log(objectofArray);
```

- **Type of object is OBJECT**

Object Traversal / Iterating over objects:

In JavaScript, traversing objects typically involves iterating over the object's properties. There are several methods to do this, each with its own use cases. Here are some common techniques:

1. for...in Loop

The for...in loop iterates over all enumerable properties of an object.

```
const obj = {  
  name: "John",  
  age: 30,  
  city: "New York"  
};  
for (let i in obj) {  
  console.log(i);           //it will print keys only name , age , city  
  console.log(obj[i]);      //it will print values only john , 30 , York  
  console.log(`${i}: ${obj[i]}`); //it will print both keys and values  
}
```

Output : name: John
age: 30

JAVASCRIPT

city: New York

2. for...of Loop with Object.entries()

Combining the for...of loop with Object.entries() allows for more readable and manageable code, especially with destructuring.

```
const obj = {  
  name: "John",  
  age: 30,  
  city: "New York"  
};  
for (let [key, value] of Object.entries(obj)) {  
  console.log(`${key}: ${value}`);  
}
```

Output : name: John
age: 30
city: New York

3. The forEach method in JavaScript is commonly used for arrays, but it can also be utilised with **objects in combination with methods like Object.keys(), Object.values(), or Object.entries()**. Here's how you can use forEach to traverse an object:

- **Using Object.keys()**

```
const obj = {  
  name: "John",  
  age: 30,  
  city: "New York"  
};
```

```
Object.keys(obj).forEach(key => {  
  console.log(`${key}: ${obj[key]}`);  
});
```

- **Using Object.values()**

```
const obj = {  
  name: "John",  
  age: 30,  
  city: "New York"  
};
```

```
Object.values(obj).forEach(value => {  
  console.log(value);  
});
```

JAVASCRIPT

- **Using Object.entries()**

```
const obj = {  
  name: "John",  
  age: 30,  
  city: "New York"  
};
```

```
Object.entries(obj).forEach(([key, value]) => {  
  console.log(`${key}: ${value}`);  
});
```

Output of All : name: John
age: 30
city: New York

Difference Between **JavaScript Array** and **JavaScript Object**.

Features	JavaScript Array	JavaScript Object
Definition	Ordered collection of elements	Collection of key-value pairs
Syntax	Defined using square brackets []	Defined using curly braces {}
Key Type	Indexed by numeric keys	Indexed by string or symbol keys
Accessing Elements	Accessed using index (e.g., array[0])	Accessed using keys (e.g., object.key or object["key"])
Order	Maintains insertion order	Does not guarantee order (insertion order is generally maintained)
Typical Use Case	Used for lists, stacks, queues	Used for representing structured data, objects with properties
Length Property	Has a length property	Does not have a length property

JAVASCRIPT

Methods	Various array-specific methods like push(), pop(), map(), filter(), etc	Object-specific methods like Object.keys(), Object.values(), Object.entries(), etc.
Iteration	Can be iterated using loops (e.g., for, forEach, map)	Can be iterated using for...in, Object.keys(), Object.values(), etc.
Memory Allocation	Optimized for element storage and access	Optimized for key-value storage
Example	let array = [1, 2, 3];	let object = {name: "John", age: 30};
Use with JSON	Represented as arrays in JSON	Represented as objects in JSON
Mutability	Elements can be added, removed, or modified	Properties can be added, removed, or modified

Pass by Value vs Pass by References :

In JavaScript, the distinction between passing by value and passing by reference is crucial to understanding how data is manipulated and shared within a program. difference between passing **objects by reference** and **by value** in JavaScript.

- In JavaScript, primitive data types like numbers and strings are passed by value, while objects are passed by reference.

Passing by value: When passing by value, a copy of the primitive value is created and passed to the function or assigned to a variable. Any changes made to the copy do not affect the original value.

Primitive data types such as **numbers, strings, booleans, null, undefined, and symbols** are passed by value.

Example:

```
function modifyValue(x) {
```

Output

JAVASCRIPT

```
x = 20;
console.log("Inside function:", x);           Inside function: 20
}
let a = 10;
//calling function
modifyValue(a);
console.log("Outside function:", a);          Outside function: 10
```

In this example, the value of a remains unchanged outside the function because the function works with a copy of a's value.

Passing by Reference : When passing by reference, a reference to the memory location of the object is passed to the function or assigned to a variable. Any changes made to the object through this reference will affect the original object. In JavaScript, **objects (including arrays and functions)** are passed by reference.

Example 1:

```
function modifyObject(obj) {
  obj.property = "new value";
  console.log("Inside function:", obj);
}
```

Outputs: Inside function: { property: 'new value' }

```
let obj = { property: "original value" };
modifyObject(obj);
console.log("Outside function:", obj);
```

Outputs: Outside function: { property: 'new value' }

Example 2:

```
function updatePerson(person) {
  // Modify the properties of the object
  person.name = "Alice";
  person.age = 30;
  console.log("Inside function:", person);
```

Outputs: Inside function: { name: 'Alice', age: 30 }

```
}
```

// Original object

Manmohan Choudhary

JAVASCRIPT

```
let person = {  
  name: "Bob",  
  age: 25  
};
```

```
console.log("Before function call:", person);
```

Outputs: Before function call: { name: 'Bob', age: 25 }

// Pass the object to the function

```
updatePerson(person);
```

```
console.log("After function call:", person);
```

Outputs: After function call: { name: 'Alice', age: 30 }

Comparison by Reference:

In JavaScript, this statement is true for objects. Two objects are considered equal (**===** or **==**) only if they reference the same memory location. Even if two separate objects have identical properties and values, they are not considered equal because they are different instances.

Here's an example to illustrate this:

```
let obj1 = { name: "Alice" };  
let obj2 = { name: "Alice" };  
let obj3 = obj1;
```

```
console.log(obj1 === obj2);    false  
console.log(obj1 == obj2);    false
```

```
console.log(obj1 === obj3);    true  
console.log(obj1 == obj3);    true
```

In this example:

- obj1 and obj2 have the same properties and values but are different instances, so obj1 === obj2 and obj1 == obj2 both evaluate to **false**.

JAVASCRIPT

- obj3 is assigned to obj1, meaning it references the same object, so obj1 === obj3 and obj1 == obj3 both evaluate to **true**.

This is different from **primitive data types** (such as strings or numbers), where equality is based on the actual value.

For example:

```
let str1 = "hello";  
let str2 = "hello";
```

```
console.log(str1 === str2);    true  
console.log(str1 == str2);     true
```

JSON (JavaScript Object Notation)

JSON (JavaScript Object Notation) is a lightweight **data interchange format** that is easy for humans to read and write, and easy for machines to parse and generate. It is based on a subset of JavaScript programming language and is commonly used for transmitting data in web applications between a server and a client.

JSON is a data interchange format derived from JavaScript objects. Objects can be easily converted to JSON and vice versa.

Key Points of JSON:

1. Syntax:

- Data is represented as key-value pairs.
- Keys are strings and should be enclosed in double quotes.
- Values can be strings, numbers, objects (JSON objects), arrays, booleans, or null.

2. Structure:

- Object: An unordered set of key-value pairs enclosed in curly braces `{}`.
- Array: An ordered list of values enclosed in square brackets `[]`

Example:

```
{
```

JAVASCRIPT

```
"name" : "John Doe",
"age": 30,
"isStudent": false,
"skills": ["HTML", "CSS", "JavaScript"],
"address": {
  "street": "123 Main St",
  "city": "Anytown",
  "zipcode": "12345"
}
}
```

Using JSON in JavaScript:

- **Parsing JSON:** To convert a JSON string into a JavaScript object, use `JSON.parse()`:

Example:

```
const jsonString = '{"name": "John", "age": 30}'; //JSON String
const obj = JSON.parse(jsonString);
console.log(obj.name);
```

Output: John

- **Stringifying JSON:** To convert a JavaScript object into a JSON string, use `JSON.stringify()`:

Example:

```
const obj = { name: "John", age: 30 };
const jsonString = JSON.stringify(obj);
console.log(jsonString);
```

Output: {"name":"John","age":30}

Use Cases:

- **Data Storage:** JSON is often used to store configuration settings and data structures in files.
- **APIs:** It is commonly used for data exchange between clients and servers in web APIs.
- **Configuration Files:** JSON is used in various configuration files, such as `package.json` for npm.

JAVASCRIPT

Example:

```
let student = {  
  id: 1,  
  sName: "Vinod",  
  sAge: 29,  
  isStudent: false,  
  greet: function () {  
    console.log(  
      `hey my id is ${student.id} & my name is ${student.sName}`  
    );  
  },  
};
```

```
let jsonData = JSON.stringify(student);  
console.log(jsonData);
```

Output : {"id":1,"sName":"Vinod","sAge":29,"isStudent":false}

```
let parsedObject = JSON.parse(jsonData);
```

```
console.log(parsedObject);
```

Output : { id: 1, sName: 'Vinod', sAge: 29, isStudent: false }

```
student.greet();
```

Output : hey my id is 1 & my name is Vinod

Benefits:

- **Human-Readable:** The syntax is easy to read and write.

JAVASCRIPT

- **Language Independent:** JSON can be used with many programming languages.
- **Lightweight:** It is a compact format, making it efficient for data exchange.

In summary, JSON is a versatile and widely used format for data interchange, particularly in web development.

Difference Between **JSON Object and JavaScript Object.**

Features	JSON Object	JavaScript Object
Definition	Data format used for exchanging data	Object in JavaScript programming
Syntax	Strict syntax: key-value pairs in double quotes	More flexible syntax: key-value pairs without quotes
Data Types	Strings, numbers, objects, arrays, booleans, null	Supports functions, dates, undefined, and more
Usage	Data interchange format	General programming and manipulation
Example	{ "name": "John", "age": 30 }	{ name: "John", age: 30 };
Parsing	Needs to be parsed with JSON.parse() in JavaScript	Directly usable as JavaScript code
Stringification	Created with JSON.stringify()	Created directly in code
Key Quotes	Mandatory double quotes around keys	Quotes around keys are optional
Trailing Commas	Not allowed (Trailing commas, also	Allowed

JAVASCRIPT

	known as final commas, are commas placed at the end of a list of items)	
Functions	Cannot include functions	Can include functions
Comments	Not allowed	Allowed

In JavaScript: Trailing commas are allowed and can be used in **arrays, objects, and function parameters**.

In JSON: Trailing commas are not allowed and will cause a syntax error if present.

12. This Keyword in js

In JavaScript, **this keyword** refers to an object. Which object depends on how this is being invoked (used or called).

The this keyword refers to different objects depending on how it is used:

In JavaScript, this keyword is a special object that refers to the context in which a function is executed. Its value depends on how and where the function is called. Understanding this keyword is crucial for managing the scope and context of functions in JavaScript.

JAVASCRIPT

Here's an explanation of how this works in different contexts:

1. **Global Context** : In the global execution context (**outside of any function**), **this** refers to the global object. In a browser, this is the **window object**.

```
console.log(this); // In a browser, this will log the Window object. {}
```

2. **Function Context** :

- **Regular Functions**: In a regular function (**not a method or an arrow function**), **this** refers to the **global object (in non-strict mode)** or **undefined (in strict mode)**.

Example:

```
function myFunction() {  
  console.log(this); // this is window object  
}  
myFunction(); // Window object (non-strict mode)  
// Window.myFunction(); // undefined (strict mode)
```

- **Methods**: When a function is called as a method of an object, **this** refers to the object the method is called on.

```
const obj = {  
  name: 'John',  
  greet: function() {  
    console.log(this.name); //this is what obj  
  } //console.log(obj.name) gives same output  
};  
obj.greet(); // 'John'
```

- **Constructor Functions**: When a function is used as a constructor with the new keyword, **this** refers to the newly created instance

```
function Person(name) {  
  this.name = name;  
}  
const person1 = new Person('Alice');  
console.log(person1.name); // 'Alice'
```

JAVASCRIPT

3. **Arrow Functions** : Arrow functions do not have their own **this** context. Instead, they inherit this from the parent scope at the time they are defined.

```
const obj = {  
  name: 'John',  
  greet: function() { //parent  
    const innerFunc = () => { //child  
      console.log(this.name);  
    };  
    innerFunc();  
  }  
};  
obj.greet();    'John'
```

4. **Event Handlers** : In event handlers, this refers to the element that received the event.

```
const button = document.querySelector('button');  
button.addEventListener('click', function() {  
  console.log(this); // Logs the button element  
});
```

5. **Explicit Binding** : You can explicitly set the value of this using **call()**, **apply()**, and **bind()** methods.

- **call()**: Invokes the function with a specified **this** value and arguments provided one by one

```
function greet() {  
  console.log(this.name);  
}  
const person = { name: 'Alice' };  
greet.call(person); // 'Alice'
```

- **apply()**: Invokes the function with a specified this value and arguments provided as an array..

```
function greet(greeting) {  
  console.log(greeting + ', ' + this.name);  
}
```

JAVASCRIPT

```
}  
const person = { name: 'Alice' };  
greet.apply(person, ['Hello']); // 'Hello, Alice'
```

- **bind()**: Returns a new function, permanently bound to a specified this value.

```
function greet() {  
  console.log(this.name);  
}  
const person = { name: 'Alice' };  
const boundGreet = greet.bind(person); //returning new function  
boundGreet(); // 'Alice'
```

Summary

- **this** in the global context refers to the global object (window in browsers).
- **this** in regular functions refers to the global object or undefined in strict mode.
- **this** in methods refers to the object the method is called on.
- **this** in constructor functions refers to the new instance created.
- **this** in arrow functions inherits from the parent scope.
- **this** in event handlers refers to the element that received the event.
- **this** can be explicitly set using **call()**, **apply()**, and **bind()** methods.

Understanding this is key to writing effective and bug-free JavaScript code.

13. New Keyword in js

The new keyword in JavaScript is used to create an instance of an object from a constructor function or a class. When you use new, it performs the following tasks:

1. **Creates a New Object**: A new empty object is created.
2. **Sets the Prototype**: The prototype of the newly created object is set to the prototype of the constructor function.
3. **Binds this**: The this keyword in the constructor function is bound to the new object.

JAVASCRIPT

4. **Returns the Object:** The new object is returned unless the constructor function explicitly returns an object.

Use Cases of new in JavaScript

1. Using with Constructor Functions:

```
function Person(name, age) {  
  this.name = name;  
  this.age = age;  
}
```

```
const person1 = new Person('John', 30);  
console.log(person1.name);  
console.log(person1.age);
```

Output: John

Output: 30

Explanation: Here, Person is a constructor function. When you create person1 using new Person(), a new object is created, and this inside Person refers to this new object. The properties name and age are set on the new object.

2. Using with ES6 Classes:

```
class Car {  
  constructor(brand, model) {  
    this.brand = brand;  
    this.model = model;  
  }  
}
```

```
const car1 = new Car('Toyota', 'Corolla');  
console.log(car1.brand);  
console.log(car1.model);
```

Output: Toyota

Output: Corolla

Explanation: Classes in JavaScript are syntactic sugar over constructor functions. The new keyword is used in the same way to instantiate objects from classes.

3. Creating Built-in Object Instances:

```
const date = new Date();  
console.log(date);
```

Output: Current Date and Time

JAVASCRIPT

```
const array = new Array(1, 2, 3);  
console.log(array);
```

Output: [1, 2, 3]

```
const regex = new RegExp('\\w+');  
console.log(regex);
```

Output: /\w+/
Choudhary-01

Explanation: The new keyword can be used with built-in constructors like Date, Array, RegExp, etc., to create instances of these built-in objects.

4. Custom Objects with Methods:

```
function Animal(type, sound) {  
  this.type = type;  
  this.sound = sound;  
}
```

```
Animal.prototype.makeSound = function() {  
  console.log(this.sound);  
};
```

```
const dog = new Animal('Dog', 'Bark');  
dog.makeSound();
```

Output: Bark

Explanation: The new keyword allows you to create objects with both properties and methods. Here, makeSound is a method added to the prototype of Animal, which can be called on instances like dog.

5. Error Handling:

```
function CustomError(message) {  
  this.name = 'CustomError';  
  this.message = message || 'Default Message';  
}  
CustomError.prototype = new Error();
```

```
throw new CustomError('Something went wrong!');
```

Explanation: You can use the new keyword to create custom error types by extending the built-in Error object.

6. ES6+ Classes: In ES6+ classes, the new keyword must be used to create instances. Trying to call a class constructor without new will result in an error.

JAVASCRIPT

```
class User {  
  constructor(name) {  
    this.name = name;  
  }  
}
```

const user = **new** User('Alice'); **Correct**
const userWithoutNew = User('Alice'); **Error: Class constructor User cannot be invoked without 'new'**

Without new: If you call a constructor function without new, this will refer to the global object (window in browsers), which can lead to unexpected results or errors. However, in strict mode, this would be undefined.

```
function Fun(name, age) {  
  this.name = name;  
  this.age = age;  
}
```

Output

```
const obj = new Fun("mohit", 23);  
console.log(obj);
```

Fun { name: 'mohit', age: 23 }

```
const obj2 = new Fun("rohit", 22);  
console.log(obj2);
```

Fun { name: 'rohit', age: 22 }

```
const obj3 = Fun("man", 24);  
console.log(obj3);
```

undefined

14. Symbol in js

In JavaScript, a **Symbol** is a **primitive data** type that was introduced in ECMAScript 6 (ES6). **Symbols are unique and immutable, making them useful for creating unique identifiers for object properties.**

Creating a Symbol :

You can create a new Symbol using the Symbol() function:

```
const mySymbol = Symbol();
```

Each call to Symbol() creates a unique symbol, even if they have the same description:

```
const sym1 = Symbol('description');
```

JAVASCRIPT

```
const sym2 = Symbol('description');
```

```
console.log(sym1 === sym2);
```

Output : false

Using Symbols as Object Properties :

Symbols can be used as unique keys for object properties, ensuring that there are no property name collisions:

```
const mySymbol = Symbol('mySymbol');
```

```
const obj = {  
  [mySymbol]: 'value'  
};
```

```
console.log(obj[mySymbol]);
```

Output : 'value'

Examples :

Output

```
let sym = Symbol("id");  
console.log(sym);  
console.log(sym.toString());  
console.log(typeof sym);  
console.log(sym.description);
```

Symbol(id)
Symbol(id)
symbol
id

//Why it is called a unique

```
let sym1 = Symbol(" im a symbol");
```

```
let sym2 = Symbol(" im a symbol");
```

```
console.log(sym1 == sym2);
```

Output false

//Symbol with Objects

```
let idSym = Symbol("id");  
let obj = {  
  name: " mohit",  
  age: 23,  
  [idSym]: 1,  
};
```

//outside the object to access symbol with object use [] not . operator

JAVASCRIPT

```
obj[idSym] = 2;
```

```
console.log(obj[idSym]);
```

Output 2

//now iterating objects property

```
for (let key in obj) {  
  console.log(key);  
}
```

Output name , age but idSym will hidden

//to access the hidden property of symbol pass it in string

```
let mySym = obj[idSym].toString();  
console.log(mySym);
```

Output : 2

Built-in Symbols :

JavaScript also includes several built-in Symbols that can be used to modify the behaviour of objects.

- **Symbol.iterator:** Used to define the default iterator for an object.
- **Symbol.toStringTag:** Used to create a string description of an object.

JAVASCRIPT

15. Set / Map in js

Set : A Set in JavaScript is a collection of unique values, Values in a Set are ordered and cannot contain duplicates. It was introduced in ECMAScript 2015 (ES6) and provides several useful methods to manage collections of data.

A Set is a built-in object in JavaScript that allows you to store unique values of any type, whether primitive values or object references.

- **Creating a Set**

To create a new Set, you can use the Set constructor:

```
// Creating an empty Set
```

```
let mySet = new Set();
```

```
// Creating a Set with initial values
```

```
let mySetWithValues = new Set([1, 2, 3, 4, 5]);
```

- **Adding Values to a Set**

You can add values to a Set using the add method:

JAVASCRIPT

```
let mySet = new Set();  
mySet.add(1);  
mySet.add(2);  
mySet.add(3);  
console.log(mySet);
```

Output: Set { 1, 2, 3 }

- **Checking for Values in a Set**

You can check if a Set contains a value using the has method:

```
console.log(mySet.has(2));  
console.log(mySet.has(4));
```

Output: true

Output: false

- **Removing Values from a Set**

You can remove values from a Set using the delete method:

```
mySet.delete(2);  
console.log(mySet);
```

Output: Set { 1, 3 }

- **Iterating Over a Set**

You can iterate over the values in a Set using a for...of loop or the forEach method:

```
let mySetWithValues = new Set([1, 2, 3, 4, 5]);
```

// Using for...of loop

```
for (let value of mySet) {  
  console.log(value);  
}
```

Output

1 2 3 4 5

// Using forEach method

```
mySet.forEach(value => {  
  console.log(value);  
});
```

- **Other Useful Methods**

size/length: Returns the number of values in the Set.

```
console.log(mySet.size);
```

Output: 2

clear: Removes all values from the Set.

JAVASCRIPT

```
mySet.clear();  
console.log(mySet);
```

Output: Set {}

- **Example: Using Set to Remove Duplicates from an Array**

```
let numbers = [1, 2, 2, 3, 4, 4, 5];  
  
let uniqueNumbers = [...new Set(numbers)];  
  
console.log(uniqueNumbers);
```

Output: [1, 2, 3, 4, 5]

- **Converting Between Set and Array**

You can easily convert between a Set and an Array.

From Array to Set : This is useful for removing duplicates:

```
let array = [1, 2, 3, 4, 4, 5];  
let uniqueSet = new Set(array);  
console.log(uniqueSet);
```

Output: Set { 1, 2, 3, 4, 5 }

From Set to Array : You can convert a Set back to an Array:

```
let arrayFromSet = [...uniqueSet];  
console.log(arrayFromSet);
```

Output: [1, 2, 3, 4, 5]

WeakSet

A WeakSet is similar to a Set in that it is a collection of unique values, but it has some important differences:

1. **Only Objects:** A WeakSet can **only contain objects**. Primitive values like numbers, strings, and booleans are not allowed.
2. **Weak References:** The objects stored in a WeakSet are held "weakly." This means that if there are no other references to an object stored in the WeakSet, that object can be garbage collected.

Example :

```
let weakSet = new WeakSet();  
let obj = { name: 'John' };  
// Add the object to the WeakSet  
weakSet.add(obj);  
console.log(weakSet.has(obj));
```

// Output: true

JAVASCRIPT

// Remove the reference to the object

obj = null;

// At this point, the object is eligible for garbage collection

Map : A **Map** in JavaScript is a collection of key-value pairs where both keys and values can be of any type. Unlike plain JavaScript objects, **Map** maintains the order of entries and allows keys to be objects, functions, or any primitive type.

- **Creating a Map** : You can create a new Map using the Map constructor:

```
let myMap = new Map();
```

You can also initialize a Map with an array of key-value pairs:

```
let myMap = new Map([  
  ['key1', 'value1'],  
  ['key2', 'value2']  
]);
```

- **Basic Methods of a Map**

1. **set(key, value)**: Adds a new element to the Map

```
myMap.set('key1', 'value1');
```

```
myMap.set('key2', 'value2');
```

2. **get(key)**: Returns the value associated with the key..

```
console.log(myMap.get('key1'));
```

//Output: value1

JAVASCRIPT

3. **has(key)**: Returns true if the key exists in the Map.
`console.log(myMap.has('key2'));` **//Output: true**
4. **delete(key)**: Removes the element with the specified key.
`myMap.delete('key1');`
`console.log(myMap.has('key1'));` **//Output: false**
5. **clear()**: Removes all elements from the Map.
`myMap.clear();`
`console.log(myMap.size);` **// Output: 0**
6. **size**: Returns the number of elements in the Map.

`console.log(myMap.size);` **// Output: 2**

- **Iterating Over a Map**

You can iterate over the keys, values, or key-value pairs in a **Map** using various methods:

1. **keys()**: Returns an iterator for the keys, using for.. Of loop

```
for (let key of myMap.keys()) {  
  console.log(key);  
}
```
2. **values()**: Returns an iterator for the values.

```
for (let value of myMap.values()) {  
  console.log(value);  
}
```
3. **entries()**: Returns an iterator for the key-value pairs.

```
for (let [key, value] of myMap.entries()) {  
  console.log(`${key}: ${value}`);  
}
```
4. **forEach()**: Executes a provided function once for each key-value pair.

`myMap.forEach((value, key) => {`

JAVASCRIPT

```
console.log(`${key}: ${value}`);  
});
```

- **Example: Using Map with different as Keys**

```
const myMap = new Map([["name", "mohit"]]);  
  
myMap.set("age", "35");  
  
myMap.set(10, "ten");  
console.log(myMap);
```

Output

Map(3) { 'name' => 'mohit', 'age' => '35', 10 => 'ten' }

- **Example: Using Map with Objects as Keys**

One of the powerful features of **Map** is that you can use objects as keys:

```
let obj1 = { id: 1 };  
let obj2 = { id: 2 };  
  
let myMap = new Map();  
myMap.set(obj1, 'Object 1');  
myMap.set(obj2, 'Object 2');
```

```
console.log(myMap.get(obj1));  
console.log(myMap.get(obj2));
```

Output: Object 1

Output: Object 2

WeakMap: A WeakMap only allows objects as keys and does not prevent them from being garbage collected if there are no other references to those objects. This makes WeakMap useful for managing memory in certain situations.

```
let weakMap = new WeakMap();  
let obj = { name: 'John' };
```

```
weakMap.set(obj, 'Some value');  
console.log(weakMap.has(obj)); // Output: true  
obj = null; // Now obj is eligible for garbage collection
```

JAVASCRIPT

Differences Between Map and Object

Features	Map	Object
Key Types	Can be any type: objects, functions, primitives	Keys are strings or symbols
Order	Maintains insertion order of keys	Does not guarantee order of keys
Size	Has a <code>size</code> property to get the number of entries	Must manually track size using <code>Object.keys().length</code>
Iteration	Iterates directly over keys, values, or entries	Iteration requires manual use of <code>for...in</code> , <code>Object.keys()</code> , etc.
Performance	Optimized for frequent additions and deletions	General-purpose, may not be optimized for specific operations
Prototype	Doesn't have default properties from a prototype (e.g., <code>toString</code>)	Inherits default properties from <code>Object.prototype</code>
Serialization	Not directly serializable using <code>JSON.stringify()</code>	Directly serializable using <code>JSON.stringify()</code>

JAVASCRIPT

Use Cases	Suitable for storing complex key-value pairs	Suitable for simple key-value storage and JSON data
-----------	--	---

16. Recursion

Recursion is a programming technique where a function calls itself in order to solve a problem. In JavaScript, recursion can be used to solve problems that can be broken down into smaller, repetitive tasks. Here's a simple example to illustrate recursion in JavaScript:

Example : Printing 1 to 10

```
function rev(n) {  
  if (n > 10) {  
    return;  
  }  
  console.log(n);  
  rev(n + 1);  
}  
rev(1);
```

Output : 1,2,3,4,5,6,7,8,9,10

Example: Factorial Calculation

The factorial of a number n (denoted as $n!$) is the product of all positive integers less than or equal to n .

For example, $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$.

JAVASCRIPT

Here's how you can calculate the factorial of a number using recursion in JavaScript:

```
function factorial(n) {  
  // Base case: if n is 0, return 1  
  if (n === 0) {  
    return 1;  
  }  
  // Recursive case: n * factorial of (n-1)  
  return n * factorial(n - 1);  
}  
console.log(factorial(5));
```

Output: 120

ADVANCED JAVASCRIPT TOPICS :

17. Synchronous JavaScript :

Synchronous JavaScript is code that is executed sequentially, line by line, from top to bottom. Each statement waits for the previous one to finish before executing.

Tasks are executed sequentially, one at a time, using a call stack. This means that each task must finish before the next one can begin. If a task takes a long time to complete, it can delay the execution of the program.

JAVASCRIPT

How it Works:

- **Blocking:** Each operation blocks the next one until it completes.
- **Simple Execution Flow:** Easier to understand and debug as the code execution follows a straightforward, top-to-bottom approach.
- **Drawbacks:** Can be inefficient, especially for tasks that take time (e.g., network requests, file reading). The browser or environment is essentially "locked up" until the operation completes, leading to a poor user experience.

Example:

```
console.log("Start");
```

```
let data = function fetchData() {  
  console.log("Assume this is a synchronous function");  
};  
console.log(data);  
console.log("End");
```

// Assume this is a synchronous function

Output :Start

**[Function: fetchData]
End**

18. Asynchronous JavaScript :

Tasks can run independently, allowing the program to continue while waiting for some tasks to finish. This is called non-blocking, and it allows multiple operations to run at the same time

Asynchronous JavaScript allows code to be executed without blocking subsequent operations. Tasks can run concurrently, and the program can continue to run other code while waiting for long-running operations to complete.

JAVASCRIPT

Due to synchronous programming sometimes important instructions get blocked due to some previous instructions which cause a delay in the UI. Asynchronous code execution allows to execute next instructions immediately and doesn't block the flow .

How it Works:

- **Non-blocking:** Uses callbacks, promises, or async/await to handle asynchronous operations.
- **Concurrent Execution:** Long-running tasks (like network requests) are executed in the background, allowing the main program to continue running.
- **Improved Performance:** Enhances responsiveness and performance by avoiding "blocking" of the main thread.

Examples:

synchronous programming

```
console.log("one");  
console.log("two");  
console.log("three");
```

Output

```
one  
two  
three
```

Asynchronous programming

```
console.log("synchronous 1");  
console.log("synchronous 2");  
console.log("synchronous 3");
```

```
function hello() {  
  console.log("delay in synchronous programming");  
}  
setTimeout(hello, 2000);           //4s = 4000ms
```

```
/*by arrow ()*/  
// setTimeout(() => {  
//   console.log("delay in synchronous programming 2");  
// }, 4000);  
console.log("synchronous 4");  
console.log("synchronous 5");
```

Output

synchronous 1

synchronous 2

JAVASCRIPT

synchronous 3

synchronous 4

synchronous 5

delay in synchronous programming 2

1. setTimeout()

It is a built-in JavaScript function used to execute a specified function or code snippet after a predetermined amount of time (in milliseconds) has elapsed. This function allows you to delay the execution of code, making it a fundamental tool for implementing timed operations or scheduling tasks in an asynchronous manner. .

Syntax : `setTimeout(function, delay, [arg1, arg2, ...]);`

- **function:** The function to be executed after the delay.
- **delay:** The time (in milliseconds) to wait before executing the specified function.
- **[arg1, arg2, ...]:** Optional additional arguments to pass to the function when it is executed.

Examples :

1. Basic Usage:

```
console.log('Start');
setTimeout(function() {
  console.log('This message is delayed by 2 seconds');
}, 2000);
console.log('End');
```

2. Passing Arguments:

Output

```
function greet(name) {
  console.log('Hello, ' + name);
}
```

Hello Alice

```
setTimeout(greet, 3000, 'Alice');
```

Here, the greet function is called with the argument 'Alice' after a 3-second delay.

JAVASCRIPT

3. Storing the Timeout ID:

```
let timeoutId = setTimeout(function() {  
    console.log('This message will be cancelled');  
}, 5000);
```

```
// Cancel the timeout  
clearTimeout(timeoutId);
```

The timeout can be cancelled before it executes by using `clearTimeout(timeoutId)`.

2. `clearTimeout()`

`clearTimeout()` is a built-in JavaScript function used to cancel a timeout that was previously established by calling `setTimeout()`. When `clearTimeout()` is called with the identifier returned by `setTimeout()`, it prevents the timeout from executing the specified function.

Syntax : `clearTimeout(timeoutID);`

- **timeoutID:** The identifier of the timeout you want to cancel. This ID is returned by the `setTimeout()` function when it is called.

Example Usage

1. Basic Example:

```
// Set a timeout  
let timeoutID = setTimeout(function() {  
    console.log('This message will not be displayed');  
}, 5000);
```

```
// Cancel the timeout  
clearTimeout(timeoutID);
```

In this example, the `clearTimeout()` function is called before the timeout completes, so the message "This message will not be displayed" is never logged to the console.

2. Conditional Timeout Cancellation:

```
let shouldCancel = true;  
let timeoutID = setTimeout(function() {  
    console.log('This message will only display if shouldCancel is false');  
}, 5000);
```

JAVASCRIPT

```
}, 3000);
```

```
if (shouldCancel) {  
  clearTimeout(timeoutID);  
}
```

Here, whether the timeout is canceled depends on the value of the `shouldCancel` variable

How It Works

When **`setTimeout()`** is called, it returns a unique identifier (a numeric ID) for the scheduled timeout. This ID can be used as a reference to cancel the timeout before it executes. The **`clearTimeout()`** function takes this ID as an argument and cancels the associated timeout if it has not yet been executed.

3.setInterval()

`setInterval` in JavaScript is a built-in function that repeatedly executes a specified function or code snippet at set time intervals. The interval between each execution is defined in milliseconds. `setInterval` falls under the **timing functions** category in JavaScript.

Syntax: `setInterval(function, delay);`

function: The function to be executed.

delay: The time (in milliseconds) to wait before each execution of the function

Example 1:normal function

```
let num = 0;  
function fun() {  
  num = num + 1;  
  console.log(num);  
}  
setInterval(fun, 2000);
```

Output 1,2,3,4.....

Example 2:using arrow function

```
setInterval(() => {  
  console.log('This message repeats every 2 seconds');  
}, 2000);
```

JAVASCRIPT

```
}, 2000); //infinite time it will execute.
```

4. clearInterval()

clearInterval in JavaScript is a function used to stop the execution of a function that was set up to repeat using **setInterval**. When you call **clearInterval**, you pass it the interval ID returned by **setInterval**, which stops the interval from continuing to execute.

Syntax: `clearInterval(intervalID);`

intervalID: The ID of the interval you want to stop, which is the value returned by `setInterval`.

Example 1:

```
// Start a repeating interval
let intervalID = setInterval(() => {
    console.log('This message repeats every 2 seconds');
}, 2000);
// Stop the interval after 10 seconds
setTimeout(() => {
    clearInterval(intervalID);
    console.log('Interval cleared');
}, 10000);
```

Output :

This message repeats every 2 seconds — 4 times print
Interval cleared

Example 2:

```
var setInterval_ID = setInterval(() => {
    let date = new Date().toLocaleTimeString();
    console.log(date);
}, 1000);

setTimeout(() => {
    clearInterval(setinterval_ID);
}, 10000);
```

Output : it print current time for 10 seconds and stop after 10 seconds

JAVASCRIPT

Key Points:

- **setInterval** continues to execute the function until it is explicitly stopped using **clearInterval**.
- If you want to execute a function only once after a delay, you should use **setTimeout** instead.

JAVASCRIPT

```
document.getElementById('btn').addEventListener('click', () => {  
    console.log("Button clicked!");  
});
```

19. Callback Function & Callback Hell

Async - await >> Promises >> Callbacks

Callback:

A callback is a function that you pass to another function as an argument. The callback function is then executed inside the outer function after a certain task is completed, usually an asynchronous task like fetching data or reading a file.

Callbacks : a callback is a function passed as argument to another function

EXAMPLE : CALLBACKS IN SYNCHRONOUS PROGRAMMING

```
function sum(a, b) {  
    console.log(a + b);  
}
```

```
function calculator(a, b, sumcallback) {  
    sumcallback(a, b);  
}
```

// 1st way → calling calculator and we can pass a function name after creation as a callback

```
calculator(1, 2, sum);
```

// 2nd way → calling calculator or we can pass full function as a argument it will also valid

JAVASCRIPT

```
function calculator(a, b, sumcallback) {  
  sumcallback(a, b);  
}
```

```
calculator(1, 4, (a, b) => {  
  console.log(a + b);  
});
```

EXAMPLE : CALLBACKS IN ASYNCHRONOUS PROGRAMMING

```
const hello = () => {  
  console.log(  
    "hello this is an example of callback in async programming by setTimeout"  
  );  
};
```

setTimeout(hello, 3000); **//it is also a callback because we are passing hello function in settime out function**

Another Example of a Callback:

```
function fetchData(callback) {  
  setTimeout(() => {  
    const data = { name: "John", age: 30 };  
    callback(data);  
  }, 1000);  
}
```

```
function processData(data) {  
  console.log(`Name: ${data.name}, Age: ${data.age}`);  
}
```

// Passing processData as a callback to fetchData
fetchData(processData);

In this example:

JAVASCRIPT

- **fetchData** is a function that simulates fetching data asynchronously (with `setTimeout`).
- **processData** is a callback function that processes the data once it's fetched.
- **fetchData(processData)** calls `fetchData` and passes `processData` as a callback. When the data is ready, `fetchData` calls `processData` with the data.

Callback Hell:

Callback hell, also known as "Pyramid of Doom," occurs when multiple asynchronous operations are nested within each other, resulting in deeply nested and hard-to-read code.

Callback hell : Nested callbacks stacked below one another forming a pyramid structure. This style of programming becomes difficult to understand & manage.

Simple Example of Callback Hell:

```
function firstTask(callback) {
  setTimeout(() => {
    console.log("First task complete.");
    callback();
  }, 1000);
}

function secondTask(callback) {
  setTimeout(() => {
    console.log("Second task complete.");
    callback();
  }, 1000);
}

function thirdTask(callback) {
  setTimeout(() => {
    console.log("Third task complete.");
    callback();
  }, 1000);
}
```

//Nested callbacks leading to callback hell

JAVASCRIPT

```
firstTask(() => {  
  secondTask(() => {  
    thirdTask(() => {  
      console.log("All tasks complete.");  
    });  
  });  
});
```

In this example:

- **firstTask**, **secondTask**, and **thirdTask** are functions that perform asynchronous tasks.
- Each task is nested within the previous task's callback, leading to deeply nested code that is hard to read and maintain.

Trying to fetch data one by one but all fetched at the same time

```
function getData(dataId) {  
  setTimeout(() => {  
    console.log("data", dataId);  
  }, 2000);  
}  
  
getData(1);  
getData(2);  
getData(3);
```

Output

data 1
data 2
data 3

EXAMPLE* TO UNDERSTAND CALLBACK HELL

```
function getData(dataId, getNextData) {  
  setTimeout(() => {  
    console.log("data", dataId);  
    if(getNextData) {  
      getNextData();  
    }  
  }, 2000);  
}
```

getData function:

- This function takes two parameters: `dataId` and `getNextData`.
- It uses `setTimeout` to simulate an asynchronous operation that takes 2 seconds.

JAVASCRIPT

- After 2 seconds, it logs "data" followed by the dataId to the console.
- If a callback function (getNextData) is provided, it is called after logging the data.

Calling getData function :

//**getData(1, getData(2))**; This is because getData(2) will be executed immediately, and its return value (which is undefined) will be passed as the second argument to getData(1).

Correct way to call function with argument as another function : this is a callback function passing using arrow function , callback hell nested function and callbacks

```
console.log("Getting data 1...");  
getData(1, () => {  
  console.log("Getting data 2...");  
  getData(2, () => {  
    console.log("Getting data 3...");  
    getData(3);  
  });  
});
```

Output :

```
Getting data 1...  
data 1  
Getting data 2...  
data 2  
Getting data 3...  
data 3
```

This code demonstrates the concept of callback functions in JavaScript. It uses nested callbacks to ensure that **getData** calls are made sequentially, with each call starting after the previous one completes. The nested structure, however, leads to what is known as "callback hell," which can make the code difficult to read and maintain.

JAVASCRIPT

20. Promises in JavaScript

Basic Promise

Promise is for “eventual” completion of task. It is an object in js. it is a solution to callback hell, Promises in JavaScript are a way to handle asynchronous operations. They represent a value that might be available now, or in the future, or never.

A Promise can be in one of three states:

1. **Pending:** Initial state, neither fulfilled nor rejected. undefined.
2. **Fulfilled:** The operation was completed successfully.
3. **Rejected:** The operation failed.

Basic Syntax : To create a promise, you use the **Promise** constructor:

```
let promise = new Promise(function(resolve, reject) {  
  // executor (the producing code)  
});
```

Promise has two handlers : both are the callbacks provided by JS.

- **resolve(res) :** is a function that is called when the asynchronous operation completes successfully.
- **reject(err) :** is a function that is called when the asynchronous operation fails.

Example :

```
let promise = new Promise((resolve, reject) => {  
  let success = true; // Simulate success or failure  
  if (success) {  
    resolve("Operation was successful!");  
  } else {  
    reject("Operation failed!");  
  }  
});  
promise.then((message) => {  
  console.log(message);  
}).catch((error) => {  
  console.log(error);  
})
```

Output : "Operation was successful!"

JAVASCRIPT

```
});
```

Example : From Start How to Create a Promise

```
let promise = new Promise((resolve, reject) => {  
  console.log("i am a promise");  
  resolve("promise is fulfilled successfully");//if promise get resolve  
  // reject("promise is rejected ");//if promise get reject  
});
```

Output : i am a promise

```
Promise {<fulfilled>: 'promise is fulfilled successfully'}  
  [[Prototype]]: Promise  
  [[PromiseState]]: "fulfilled"  
  [[PromiseResult]]: "promise is fulfilled successfully"
```

****Example of Promise in API by help of function** simple Example**

```
function getData2(dataId, getNextData) {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      console.log("data", dataId);  
      resolve("success");  
      // reject("throwing a error");  
      if (getNextData) {  
        getNextData();  
      }  
    }, 5000);  
  });  
}
```

Output : if the promise get resolved successfully

```
let promisecall = getData2(123);  
// we are calling function that returns a promise
```

undefined

promisecall // **Checking the State of Promise**

```
Promise {<pending>}
```

JAVASCRIPT

```
[[Prototype]]: Promise  
[[PromiseState]]: "pending"  
[[PromiseResult]]: undefined
```

data 123 // **getting data from function which is return by Promise**

promisecall // **Again checking the State of Promise**

```
Promise {<fulfilled>: 'success'}  
  [[Prototype]]: Promise  
  [[PromiseState]]: "fulfilled"  
  [[PromiseResult]]: "success"
```

Output : if the promise get rejected fails

```
let promisecall = getData2(123);  
// we are calling function that returns a promise
```

undefined

promisecall // **Checking the State of Promise**

```
Promise {<pending>}  
  [[Prototype]]: Promise  
  [[PromiseState]]: "pending"  
  [[PromiseResult]]: undefined
```

Uncaught (in promise) error // **getting error**

promisecall // **Again checking the State of Promise**

```
Promise {<rejected>: 'error'}  
  [[Prototype]]: Promise  
  [[PromiseState]]: "rejected"  
  [[PromiseResult]]: "error"
```

Now in real programming we are not creating promises , we just need to understand and handle them from their api's and systems that return the promise.

JAVASCRIPT

.then() and .catch() in Promises

Promises provide two main methods to handle the completion of asynchronous operations: **.then()** and **.catch()**.

.then()

The **.then()** method is used to handle the successful completion of a promise. It takes up to two arguments:

1. **onFulfilled**: A function that gets called if the promise is fulfilled.
2. **onRejected (optional)**: A function that gets called if the promise is rejected.
3. **Syntax** : **promise.then(onFulfilled, onRejected);//res**
4. **Example** :

```
let promise = new Promise((resolve, reject) => {  
  let success = true; // Simulate success or failure  
  if (success) {  
    resolve("Operation was successful!");  
  } else {  
    reject("Operation failed!");  
  }  
});  
promise.then(  
  (message) => {  
    console.log(message); // "Operation was successful!"  
  },  
  (error) => {  
    console.log(error); // "Operation failed!"  
  }  
);
```

.catch()

The **.catch()** method is used to handle errors (rejections) in promises. It is a shorthand for **.then(null, onRejected)** and is often used for better readability.

Syntax : **promise.catch(onRejected);//err**

Example :

```
let promise = new Promise((resolve, reject) => {  
  let success = false; // Simulate success or failure
```

JAVASCRIPT

```
if (success) {  
    resolve("Operation was successful!");  
} else {  
    reject("Operation failed!");  
}  
});
```

```
promise.then((message) => {  
    console.log(message); // This won't run since success is false  
})  
.catch((error) => {  
    console.log(error); // "Operation failed!"  
});
```

Example : Now how to use Promise - simple Example

```
const getPromise = () => {  
    return new Promise((resolve, reject) => {  
        console.log("I am a promise in javascript");  
        resolve("succes");  
        // reject("network error");  
    });  
};
```

//it runs when promise get resolved

```
let promise_Ans = getPromise();  
promise_Ans.then((res) => {  
    console.log("promise fulfilled", res);  
});
```

Output

I am a promise in javascript
promise fulfilled success

//it runs when promise get rejected

```
promise_Ans.catch((err) => {  
    console.log("promise rejected", err);  
});
```

I am a promise in javascript
promise rejected network error

JAVASCRIPT

Promise Chaining

Promise chaining is a technique where you perform a series of asynchronous operations one after another, using the result of one operation as the input to the next. This is done by returning a new promise in the **.then** method.

Example 1

```
function asyncOperation1() {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve("Result of operation 1");
    }, 1000);
  });
}

function asyncOperation2(previousResult) {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve(`${previousResult}, and result of operation 2`);
    }, 1000);
  });
}

function asyncOperation3(previousResult) {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve(`${previousResult}, and result of operation 3`);
    }, 1000);
  });
}

asyncOperation1().then(result1 => {
  console.log(result1); // Result of operation 1
  return asyncOperation2(result1);
})
.then(result2 => {
  console.log(result2); // Result of operation 1, and result of operation 2
  return asyncOperation3(result2);
})
.then(result3 => {
  console.log(result3); // Result of operation 1, and result of operation 2, and result of operation 3
})
.catch(error => {
  console.error(error);
});
```

JAVASCRIPT

****IMP PROMISE CHAIN - simple example:****

Example 2

```
function asyncFunc1() {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      console.log("some data1");  
      resolve("success");  
    }, 4000);  
  });  
}
```

```
function asyncFunc2() {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      console.log("some data2");  
      resolve("success");  
    }, 4000);  
  });  
}
```

//we are calling functions with promises - **parallelly;**

Output

```
console.log("fetching data1.....");  
let p1 = asyncFunc1();  
p1.then((res) => {  
  console.log(res);  
});
```

```
console.log("fetching data2.....");  
let p2 = asyncFunc2();  
p2.then((res) => {  
  console.log(res);  
});
```

```
fetching data1.....  
fetching data2.....  
some data1  
success  
some data2  
success
```

In this both promises are called same time and give output at same time

JAVASCRIPT

Solution is NESTED PROMISES OR PROMISE CHAIN

```
console.log("fetching data1.....");
```

```
let p1 = asyncFunc1();  
p1.then((res) => {  
  console.log(res);  
}
```

Output :

fetching data1.....
some data1
success

```
console.log("fetching data2.....");  
let p2 = asyncFunc2();  
p2.then((res) => {  
  console.log(res);  
}
```

fetching data2.....
some data2
success

```
console.log("fetching data2 again.....");  
let p3 = asyncFunc2();  
p3.then((res) => {  
  console.log("successfully done");  
  });  
});  
});
```

fetching data2 again.....
some data2
successfully done

Short trick

```
console.log("fetching data 1.....");  
asyncFunc1().then((res) => {  
  console.log(res);  
  console.log("fetching data 2.....");  
  asyncFunc2().then((res) => {  
    console.log(res);  
  });  
});  
});
```

JAVASCRIPT

ACTUAL PROMISE CHAIN : EXAMPLE:

```
function asyncFunc1() {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      console.log("some data1");  
      resolve("success 1");  
    }, 4000);  
  });  
}  
function asyncFunc2() {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      console.log("some data2");  
      resolve("success 2");  
    }, 4000);  
  });  
}  
function asyncFunc3() {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      console.log("some data3");  
      reject("error..");  
    }, 4000);  
  });  
}
```

```
// Actual Promise Chaining  
console.log("fetching data1.....");  
asyncFunc1()  
  .then((res) => {  
    console.log(res);  
    console.log("fetching data2.....");  
    return asyncFunc2();  
  })  
  .then((res) => {  
    console.log(res);  
    console.log("fetching data3.....");  
    return asyncFunc3();  
  })  
  .then((res) => {  
    console.log(res);  
    console.log("successfully done");  
  })  
  .catch((error) => {  
    console.error(error);  
  });
```

Output
fetching data1.....
some data1
success 1

fetching data2.....
some data2
success 2

fetching data3.....
some data 3
error

JAVASCRIPT

so the work that we did using nested callbacks we then also did it by promises chain..

REVISITING THE CALLBACK HELL EXAMPLE : SOLVING IT USING PROMISES

```
function getData(dataId) {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      console.log("data", dataId);  
      resolve("successful");  
    }, 2000);  
  });  
}
```

//USING PROMISE CHAIN:

```
console.log("getting data 1...");  
let p1 = getData(123);  
p1.then((res) => {  
  console.log(res);  
  
  let p2 = getData(234);  
  console.log("getting data 2...");  
  p2.then((res) => {  
    console.log(res);  
  });  
});
```

//short trick

```
console.log("getting data 1..");  
getData(123).then((res) => {  
  console.log(res);  
  console.log("getting data 2..");  
  getData(234).then((res1) => {  
    console.log(res1);  
  });  
});
```

Output

```
getting data 1...  
data 123  
successful
```

```
getting data 2.  
data 234  
successful
```

JAVASCRIPT

//Actual promise chain

```
console.log("getting data 1...");
getData(123)
  .then((res) => {
    console.log("getting data 2...");
    return getData(234);
  })
  .then((res) => {
    console.log("getting data 3...");
    return getData(345);
  })
  .then((res) => {
    console.log(res);
  });
```

Output

```
getting data 1...
data 123
getting data 2...
data 234
getting data 3...
data 345
successful
```

Drawback of Promises

Promises can lead to complex and hard-to-read code, especially with deep chains and nested `.then()` calls, making error handling and debugging more challenging.

JAVASCRIPT

21. Async - Await in js

async and **await** are syntactic features in JavaScript that make working with asynchronous code more readable and easier to manage. They are used with Promises to handle asynchronous operations.

- **async Functions** :A function declared with **async** always returns a **Promise**. It allows you to write asynchronous code in a more readable way.

Syntax : **async** function myFunction() {
 // function b; }

- **await Operator** : The await operator is used to wait for a Promise to resolve or reject within an async function. When await is used, the async function pauses until the Promise settles and then resumes with the resolved value or throws an error if the Promise is rejected.
- **await Operator used inside async functions to pause the execution until a Promise is resolved.**

Syntax : let result = **await** promise;

Example 1 : Here's a basic example to demonstrate async and await:

```
function fetchData() {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      resolve('Data fetched');  
    }, 2000);  
  });  
}
```

```
async function getData() {  
  console.log('Fetching data...');  
  let data = await fetchData();  
  console.log(data);  
  console.log("execution ends");  
}
```

getData();

Output

Fetching data...
Data fetched
execution ends

JAVASCRIPT

In this example:

1. **fetchData** is a function that returns a Promise. It simulates fetching data asynchronously by using `setTimeout`.
2. **getData** is an async function that calls `fetchData` and waits for it to resolve using `await`. While **fetchData** is waiting to resolve, **getData** is paused.
3. When **fetchData** resolves, **getData** resumes and logs the fetched data.

Error Handling

You can use `try...catch` blocks within async functions to handle errors:

async function `getData()` {

try {

 let data = await `fetchData()`;

 console.log(data);

 } **catch** (error) {

 console.error('Error:', error);

 }

}

Output

Data fetched

EXAMPLE OF ASYNC - AWAIT FROM START :

ASYNC AWAIT IS A FUNCTION OR METHOD THAT IS USED IN PLACE OF CALLBACKS , PROMISES CHAIN:

Example 2 :

function **api()** {

 return new Promise((resolve, reject) => {

 setTimeout(() => {

 console.log("weather data");

 resolve(200);

 }, 2000);

 });

} //it replicate the then and catch functions of promised chain

async function getWeatherData() {

 /* EASY TO BREAK ONE CALL AFTER ANOTHER*/

await api(); //1st call;

await api();//2nd call;

}

getWeatherData();

Output

weather data

weather data

JAVASCRIPT

Example 2 : await is only use with async functions

```
function getData(dataId) {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      console.log("data", dataId);  
      resolve("successful");  
    }, 2000);  
  });  
}
```

//ASYNC - AWAIT

```
async function getAllData() {  
  console.log("getting data1....");  
  await getData(1);  
  console.log("getting data2....");  
  await getData(2);  
  console.log("getting data3....");  
  await getData(3);  
}  
getAllData();
```

Output

```
getting data1....  
1  
getting data2....  
2  
getting data1....  
3
```

There is an unnecessary function call to get the data by async - await.

SOLUTION : IIFE - IMMEDIATELY INVOKED FUNCTION EXPRESSION.

IIFE is a function that is called immediately as soon as, it has no name , it is defined and is one time used and executed.**mainly used in api calls and one time execution.**

//Using IIFE

```
(async function () {  
  console.log("getting data1....");  
  await getData(1);  
  console.log("getting data2....");  
  await getData(2);  
  console.log("getting data3....");  
  await getData(3);  
})();
```

//used to avoid unnecessary variables to execute an async function.

JAVASCRIPT

Example 3 : To understand the difference between Callback - hell , Promise - chain and Async - await.

1st - Callback

```
function getData(dataId, getNextData) {  
  setTimeout(() => {  
    console.log("data", dataId);  
    if (getNextData) {  
      getNextData();  
    }  
  }, 2000); }
```

```
getData(1, () => {  
  console.log("getting data 2...");  
  getData(2, () => {  
    console.log("getting data 3...");  
    getData(3, () => {  
      console.log("getting data 4...");  
      getData(4);  
    });  
  });  
});
```

2nd - Promise chain

```
function getData(dataId) {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      console.log("data", dataId);  
      resolve("successful");  
    }, 2000);  
  });  
}
```

```
console.log("getting data 1...");  
getData(1)  
  .then((res) => {  
    console.log("getting data 2...");  
    return getData(2);  
  })
```

JAVASCRIPT

```
.then((res) => {  
  console.log("getting data 3...");  
  return getData(3);  
})  
.then((res) => {  
  console.log(res);  
});
```

3rd - Async - await

```
(async function () {  
  console.log("getting data 1...");  
  await getData(1);  
  console.log("getting data 2...");  
  await getData(2);  
  console.log("getting data 3...");  
  await getData(3);  
})();
```

Output :

```
getting data 1...  
data 1  
getting data 2...  
data 2  
getting data 3...  
data 3
```

JAVASCRIPT

22. call, bind, and apply

In JavaScript, **call**, **bind**, and **apply** are methods that allow you to control the value of **this** when invoking functions. They are particularly useful when working with methods in objects or when borrowing methods from one object to use in another.

1. call()

Purpose: Invokes a function with a given **this** value and arguments provided individually.

Syntax: `functionName.call(thisArg, arg1, arg2, ...)`

Example:

```
function greet(greeting, punctuation) {  
    console.log(greeting + ', ' + this.name + punctuation);  
}  
const obj1 = { name: 'John' };  
const obj2 = { name: 'mike' };
```

```
greet.call(obj2, 'Hello', '!');
```

Output: Hello, mike!

2. apply()

Purpose: Similar to **call()**, but arguments are provided as an array.

Syntax: `functionName.apply(thisArg, [arg1, arg2, ...])`

Example:

```
function greet(greeting, punctuation) {  
    console.log(greeting + ", " + this.name + punctuation);  
}
```

```
const obj1 = {name: "John"};  
const obj2 = {name: "mike"};  
greet.apply(obj1, ["Hello", "!"]);
```

Output: Hello, John!

3. bind()

- **Purpose:** Creates a new function that, when called, has its **this** value set to the provided **thisArg**, with a given sequence of arguments preceding any provided when the new function is called.

JAVASCRIPT

- **Syntax:** `const boundFunction = functionName.bind(thisArg, arg1, arg2, ...)`

Example:

```
function greet(greeting, punctuation) {  
  console.log(greeting + ", " + this.name + punctuation);  
}  
const person = {name: "John"};
```

```
const greetJohn = greet.bind(person, "Hello");  
console.log(typeof greetJohn);  
console.log(greetJohn);  
greetJohn('!');
```

Output

```
function  
[Function: bound greet]  
Hello, John!
```

Simple Example for All :

```
function fun(age, gender) {  
  console.log(this.fname, age, gender);  
}  
obj1 = {  
  fname: "mohit",  
};  
obj2 = {  
  fname: "rohit",  
};
```

```
fun.call(obj1, 12, "M");  
fun.apply(obj2, [12, "M"]);  
var a = fun.bind(obj2, [12, "M"]);  
console.log(a);  
console.log(typeof a);  
a(".");
```

Output

```
mohit 12 M  
rohit 12 M  
  
[Function: bound fun]  
function  
rohit [ 12, 'M' ] .
```

JAVASCRIPT

Object Oriented Programming

23. Proto & ProtoType

A javascript object is an entity having state and behavior(properties and methods).

JS objects have a special property called prototype, which is an object.

We can set prototype using `__proto__`

***If Object & prototype have the same method, the object's method will be used.**

In JavaScript, **proto** and **prototype** are two different concepts related to how objects are linked and how inheritance works. Here's a breakdown of each:

1. `__proto__`

- Every JavaScript object has a hidden property called `__proto__`.
- This property points to another object, which is the prototype from which the original object can inherit properties and methods.
- Think of it as a link that connects an object to its "parent" object from which it can borrow things.
- When you try to access a property or method on an object, JavaScript first looks for it on the object itself. If it doesn't find it there, it looks at the object's `__proto__`, then at the `__proto__` of that prototype, and so on, until it finds the property or reaches the end of the prototype chain (usually **`Object.prototype`**).

Example 1 : kind of inheritances

```
const animal = {  
  eats: true  
};
```

```
const rabbit = {  
  jumps: true  
};
```

```
rabbit.__proto__ = animal; // rabbit inherits from animal  
console.log(rabbit.eats); // true (rabbit gets this from animal)
```

JAVASCRIPT

Output :

true

rabbit

{jumps: true}

jumps : true

[[Prototype]] : Object

eats : true //inherit properties of animal object

Example 2 : using `__proto__`

Output

```
var obj = {
```

```
  fName: "John",
```

```
};
```

```
var obj2 = {
```

```
  lName: "Smith",
```

```
};
```

```
obj2.__proto__ = obj;
```

```
console.log(obj2.fName);
```

John

Example 3 : using `Object.create()` method

```
var obj1 = {
```

```
  fName: "john",
```

```
};
```

```
var obj3 = Object.create(obj1);
```

```
obj3.lName = "Smith";
```

```
console.log(obj3.lName);
```

Smith

```
console.log(obj3.fName);
```

John

`__proto__` is like a shortcut that an object uses to check if it can borrow some property or method from another object.

JAVASCRIPT

2. Prototype:

- **prototype** is a property found in constructor functions (functions used to create objects).
- It's like a blueprint that tells the newly created objects what properties and methods they should have.
- The **prototype** object is where you define properties and methods that should be shared by all instances of objects created by that constructor function.
- The **__proto__** of the instances created by the constructor function points to the **prototype** of that constructor.

Example 1: this also an example of proto with constructor function.

```
function Person(name, age) {//constructor function
```

```
    let newPerson = Object.create(obj);  
    newPerson.name = name;  
    newPerson.age = age;  
    return newPerson;  
}  
var obj = {  
    greet() {  
        console.log(`Hello ${this.name}`);  
    },  
};
```

```
let user = Person("john", 18);  
console.log(user.age);//age is found in newPerson object  
user.greet(); //greet function is found in obj reference of newPerson object  
console.log(user);
```

18

Hello john

{ name: 'john', age: 18 }

Example 2: using prototype and new keyword

```
function Person(name, age) {  
    // let newPerson = Object.create(Person.prototype);  
    this.name = name;
```

JAVASCRIPT

```
    this.age = age;
  }
  Person.prototype.greet = function () {
    console.log(`Hello ${this.name}`);
  };
};
```

let user = **new** Person("john", 18); // **using new keyword can create a new object**

```
console.log(user.age);
user.greet(); // prototype is used to take reference
console.log(user);
18
Hello john
Person { name: 'john', age: 18 }
```

IMPORTANT

Yes, classes in JavaScript are essentially a more convenient and modern syntax for working with prototypes. While the underlying mechanism of JavaScript objects and inheritance is still based on prototypes, classes provide a cleaner and more intuitive way to define and work with objects and their inheritance.

Before **classes** were introduced in ES6 (ECMAScript 2015), JavaScript used prototypes for inheritance. Every object in JavaScript has a hidden property called **__proto__**, which is a reference to another object (the prototype). When you access a property or method on an object, JavaScript first looks for it on the object itself. If it doesn't find it, it looks at the object's prototype, and so on, up the chain.

24. Class and Object in js

In JavaScript, a **class** is a **blueprint** for creating objects that share the same properties and methods. It's a way to organize your code by grouping related

JAVASCRIPT

functions (methods) and variables (properties) into a single structure. This makes your code more modular, reusable, and easier to maintain.

When to Use Classes

- **When you need multiple similar objects:** If you're creating several objects that share common properties and methods, a class can help you avoid repeating code.
- **When you want to organize code better:** Classes help encapsulate related functionality in one place, making your code cleaner.
- **When you need inheritance:** Classes allow you to create a base class with common functionality and then extend it to create more specific classes.

How to Use Classes

1. Defining a Class : You define a class using the **class** keyword:

```
class Animal {  
  // Constructor method  
  constructor(name, species) {  
    this.name = name; // Property  
    this.species = species; // Property  
  }  
  // Method  
  speak() {  
    console.log(`${this.name} makes a sound.`);  
  }  
}
```

2. Creating an Instance : Once a class is defined, you can create objects (instances) from it:

```
const dog = new Animal('Buddy', 'Dog');  
dog.speak(); // Output: Buddy makes a sound.
```

3. Adding Methods : You can add methods (functions) inside the class, and they will be available on all instances of the class:

```
class Animal {  
  constructor(name, species) {  
    this.name = name;
```

JAVASCRIPT

```
    this.species = species;
  }

  speak() {
    console.log(`${this.name} makes a sound.`);
  }

  describe() {
    console.log(`${this.name} is a ${this.species}.`);
  }
}
```

```
const cat = new Animal('Whiskers', 'Cat');
cat.describe();
```

Output: Whiskers is a Cat.

Example of class with multiple objects.

```
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
  greet() {
    console.log(`Hello ${this.name} your age is ${this.age}`);
  }
}

let obj1 = new Person("mohit", 23);
obj1.greet();
```

Hello mohit your age is 23

```
let obj2 = new Person("rohit", 22);
obj2.greet();
```

Hello rohit your age is 22

```
console.log(typeof Person);
console.log(Person.prototype.greet);
```

**function
[Function: greet]**

4. Inheritance : Classes support inheritance, allowing you to create a new class based on an existing one:

```
class Animal {
  constructor(name) {
    this.name = name;
  }
}
```

JAVASCRIPT

```
}  
  speak() {  
    console.log(`${this.name} makes a noise`);  
  }  
}  
class Dog extends Animal {  
  constructor(name) {  
    super(name); //calling the animal class constructor  
  }  
  walk() {  
    console.log(`${this.name} walking..`);  
  }  
}
```

```
const animal1 = new Dog("jocky", 20);  
animal1.speak();  
animal1.walk();
```

Output

```
jocky makes a noise  
jocky walking..
```

Key Points

Constructor:

- **What it is:** A special method in a class used to initialize new objects.
- **How it works:** Runs automatically when a new instance of the class is created

```
class Person {  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
}  
const john = new Person('John', 30);
```

Methods: Functions that belong to a class and can be called on instances.

Super:

- **What it is:** A keyword used to call the constructor or methods of a parent class.
- **How it works:** It must be called in a subclass constructor before accessing **this**.

JAVASCRIPT

```
class Dog extends Animal {  
  constructor(name, breed) {  
    super(name); // Calls the parent constructor  
    this.breed = breed;  
  }  
}
```

Inheritance:

- **What it is:** A mechanism that allows a class (subclass) to inherit properties and methods from another class (superclass).
- **How it works:** Use the **extends** keyword to create a subclass that inherits from a parent class.
- If Child & Parent have the same method , the child's method will be used.**[Method Overriding]**

```
class Animal {  
  speak() {  
    console.log("This animal makes a sound.");  
  }  
}
```

```
class Dog extends Animal {  
  speak() {  
    console.log("The dog barks.");  
  }  
}  
  
const myDog = new Dog();  
myDog.speak();
```

Output: The dog barks.

Examples:

1. Basic example of class

```
class ToyotaCar {  
  start() {  
    console.log("start");  
  }  
  stop() {  
    console.log("end");  
  }  
}
```

JAVASCRIPT

```
}
setBrand(brand) {
  this.brand = brand;
}
showBrand() {
  console.log(`brand name is ${this.brand}`);
}
}

const fortuner = new ToyotaCar();
fortuner.start();           start
fortuner.setBrand("Maruti"); brand name is Maruti
fortuner.showBrand();       end
fortuner.stop();

const XUV = new ToyotaCar();
XUV.start();               start
XUV.setBrand("Hero");      brand name is Hero
XUV.showBrand();           end
XUV.stop();
```

2. Basic Example of class with Constructor

```
class ToyotaCar {
  constructor(brand) {
    this.brand = brand;
  }
  start() {
    console.log("start");
  }
  stop() {
    console.log("end");
  }
  showBrand() {
    console.log(`brand name is ${this.brand}`);
  }
}

const fortuner = new ToyotaCar("Maruti");
fortuner.start();           start
fortuner.showBrand();       brand name is Maruti

const XUV = new ToyotaCar();
```

JAVASCRIPT

XUV.start();

start

XUV.showBrand();

brand name is undefined

3. Full Inheritance & Super keyword with Example

```
class Parent {
  constructor(name) {
    this.name = name;
  }
  eat() {
    console.log(`${this.name} is eating..`);
  }
  sleep() {
    console.log(`${this.name} is sleeping..`);
  }
}

class Engineer extends Parent {
  constructor(name, branch) {
    super(name); //to call parent class constructor
    this.branch = branch;
  }
  work() {
    super.sleep(); // also to access parent class methods in child class
    console.log(
      "creating problems and solving them efficiently >> " + this.branch
    );
  }
}

let a = new Engineer("Mohit", "IT");
a.eat(); //Mohit is eating.. //Mohit is sleeping..
a.work();//creating problems and solving them efficiently >> IT
```

Static Keyword

The **static** keyword in JavaScript is used to define **static methods** or properties in a class. These methods and properties belong to the class itself rather than to instances of the class. This means you can call a static method or access a static property directly on the class without creating an instance of it.

Why Use static?

JAVASCRIPT

- **Utility Functions:** When you need a function that's related to a class but doesn't depend on the instance data, you can define it as a static method.
- **Shared Constants:** You can use static properties to store constants or other values that should be shared across all instances.

How to Use static

1. Static Methods : Static methods are useful for utility functions or calculations that don't require instance-specific data.

```
class MathHelper {  
  // Static method  
  static add(a, b) {  
    return a + b;  
  }  
}
```

//Call the static method directly on the class

```
console.log(MathHelper.add(5, 3));
```

Output: 8

Here, add is a static method that can be called directly using the class name MathHelper without creating an instance.

2. Static Properties : Static properties are shared across all instances of the class, meaning they belong to the class itself.

```
class Settings {  
  // Static property  
  static siteName = 'My Awesome Site';  
  static getSiteName() {  
    return Settings.siteName;}}
```

// Access the static property directly from the class

```
console.log(Settings.siteName);
```

Output: My Awesome Site

```
console.log(Settings.getSiteName());
```

Output: My Awesome Site

Example 1 :

```
class Person {  
  constructor(name) {  
    this.name = name;  
  }  
  speak() {  
    console.log(`${this.name} makes a noise`);  
  }  
}
```

JAVASCRIPT

```
}  
static sing(name) {  
  console.log(`${name} is singing a song`);  
}  
}
```

```
let people = new Person("mohit");  
people.speak();  
//people.sing();  
Person.sing(people.name);
```

mohit makes a noise
error because sing is static method
mohit is singing a song

Example 2 :

```
class Game {  
  static score = 0;  
  
  startGame() {  
    console.log(`game is starting...`);  
  }  
  
  endGame() {  
    console.log(`game is ending...`);  
    Game.updateScore();  
  }  
  
  static updateScore() { it is not access by object of class  
    Game.score++;  
    console.log(`Score : ${Game.score}`);  
  }  
}
```

```
let player1 = new Game();  
player1.startGame();  
player1.endGame();
```

game is starting...
game is ending...
score : 1

Public and Private

In JavaScript, **private** and **public** are concepts related to controlling the visibility and accessibility of properties and methods in object-oriented programming (OOP). They define who can access or modify certain parts of a class.

1. Public

JAVASCRIPT

- **What it is:** Public properties and methods are accessible from anywhere—both inside and outside the class. By default, all properties and methods in a JavaScript class are public.
- **When to use:** Use public properties and methods when you want them to be accessible by other parts of your program or by instances of the class.
- **Example:**

```
class Person {  
  // Public property  
  name;  
  
  constructor(name) {  
    this.name = name; // Public property  
  }  
  
  // Public method  
  greet() {  
    console.log(`Hello, my name is ${this.name}`);  
  }  
}
```

```
const person = new Person('Alice');  
console.log(person.name); // Accessible  
person.greet(); // Accessible
```

Alice
Hello, my name is Alice

2. Private

- **What it is:** Private properties and methods are accessible only within the class they are defined in. They cannot be accessed or modified directly from outside the class. In JavaScript, private members are defined using the # symbol.
- **When to use:** Use private properties and methods when you want to hide implementation details and protect the internal state of an object from being accessed or modified directly.
- **Example:**

```
class Person {  
  // Private property  
  #age;  
  
  constructor(name, age) {
```

JAVASCRIPT

```
this.name = name; // Public property
this.#age = age; // Private property
}

// Private method
#calculateBirthYear() {
  const currentYear = new Date().getFullYear();
  return currentYear - this.#age;
}

// Public method
getDetails() {
  console.log(`Name: ${this.name}, Birth Year:
  ${this.#calculateBirthYear()}`);
}
}

const person = new Person('Bob', 30);

console.log(person.name); // Accessible
person.getDetails(); // Accessible
// console.log(person.#age); // Error: Private field '#age' must be
// declared in an enclosing class
// person.#calculateBirthYear(); // Error: Private field
// '#calculateBirthYear' must be declared in an enclosing class
```

try catch throw block in js

In JavaScript, **try**, **catch**, and **throw** are used for **error handling**. They help you manage errors gracefully, rather than letting them crash your program. Here's a breakdown of each:

1. try Block : The try block contains code that might throw an error. JavaScript will "try" to execute the code inside this block.

Why to use: To test a block of code and anticipate potential errors without crashing the entire program.

JAVASCRIPT

2. catch Block : If an error occurs in the try block, the catch block will execute. It receives the error object, which contains information about what went wrong.

Why to use: To handle errors gracefully, such as by logging them, displaying a user-friendly message, or taking corrective action.

3. throw Statement : The throw statement is used to create your own custom errors. When you throw an error, the code inside the try block stops executing, and control is passed to the catch block.

Why to use: To signal an error condition yourself, especially when you need to enforce certain rules or validations.

Example :

```
function divide(a, b) {  
  try {  
    if (b == 0) {  
      throw new Error("can't divide by 0");  
    } else {  
      console.log(`division of a and b is ${a / b}`);  
    }  
  } catch (err) {  
    console.log(err.message);  
  }  
  console.log("program is ended..");  
}  
divide(10, 2);  
divide(10, 0);
```

Output

```
division of a and b is 5  
program is ended..  
can't divide by 0  
program is ended..
```

25. DOM in Js

What is the DOM in JavaScript?

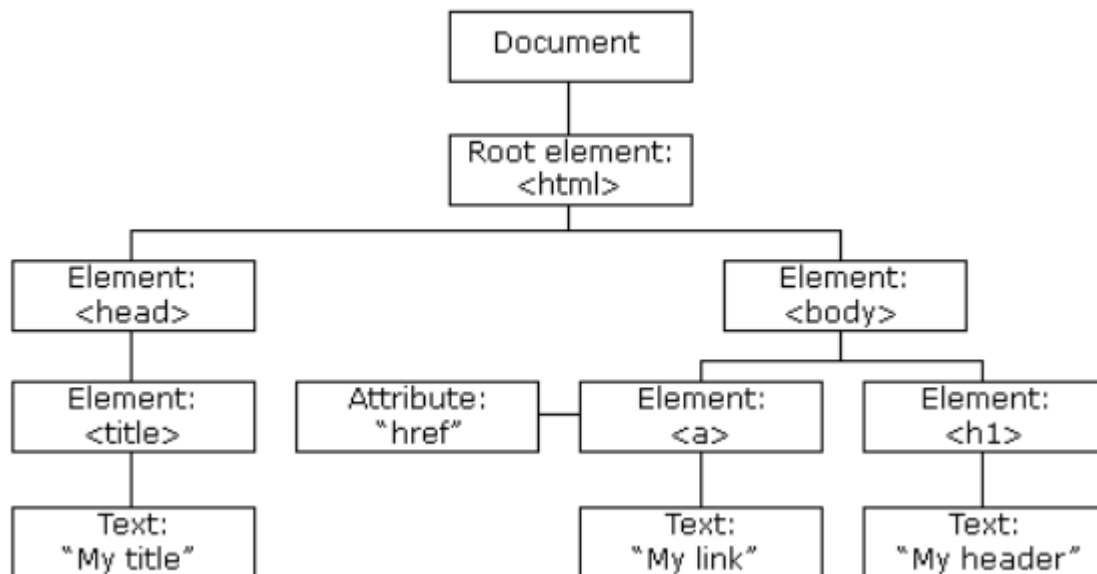
The **Document Object Model (DOM)** is a programming interface for web documents. It represents the structure of a web page as a hierarchical tree of **objects (or nodes)**, where each node corresponds to a part of the document. These nodes can represent **HTML elements, attributes, text, or comments**.

It has properties and methods. By the help of document objects, we can add dynamic content to our web page.

JAVASCRIPT

In simpler terms, when a web page is loaded, the browser converts the HTML into a **structured tree of objects (the DOM)**. This tree-like structure allows developers to interact with and manipulate the content, structure, and styling of the web page using JavaScript.

When a web page is loaded, the browser creates a Document Object Model (DOM) of the page.



Why is the DOM used?

The DOM is used for various reasons:

1. **Dynamic content manipulation:** JavaScript can be used to modify HTML elements, attributes, and styles after the page has loaded. This makes it possible to create dynamic, interactive, and responsive web pages without needing to reload the page.
2. **Event handling:** The DOM allows developers to attach event listeners (e.g., clicks, keyboard presses, mouse movements) to specific elements on the page. This makes user interactions possible and enables the building of rich web applications.
3. **Accessing HTML elements:** With the DOM, JavaScript can access and manipulate any element or attribute within the HTML document. This is essential for tasks like form validation, content updates, and interactive user interfaces.
4. **Modifying the structure:** The DOM allows you to add, remove, or rearrange elements in the document. For example, you can dynamically

JAVASCRIPT

add new items to a list, remove an element when it's no longer needed, or change the order of elements.

How the DOM is used in JavaScript

JavaScript provides several properties & methods to interact with and manipulate the DOM.

DOM Manipulation

1. Accessing Elements:

You can use various methods to select elements in the DOM.

- **getElementById**: Selects an element by its unique ID. It returns the element.
- **getElementsByClassName**: Selects all elements with a certain class. It returns an indexed **HTML Collection**.
- **getElementsByTagName**: Selects all elements with a specific tag name (e.g., div, p, etc.). it returns **HTML Collection**.
- **querySelector**: Selects the first element that matches a CSS selector.
- **querySelectorAll**: Selects all elements that match a CSS selector. It returns an indexed **NodeList**.

Example:

```
// Accessing an element by ID
const heading = document.getElementById("main-heading");

// Accessing an element by Tag Name
const heading = document.getElementsByTagName("p");

// Accessing elements by class
const items = document.getElementsByClassName("list-item");

// Accessing elements using querySelector
const button = document.querySelector(".submit-btn");
```

JAVASCRIPT

```
// Accessing elements using querySelectorAll  
const button = document.querySelectorAll(".list-item");
```

HTMLCollection : it is a collection of HTML elements that are retrieved from the DOM using methods like **getElementsByTagName**, **getElementsByClassName**, or **children**. It behaves like an array but is not a true array, meaning you can access elements by index (e.g., `collection[0]`), but it doesn't have array methods like `forEach`, `map`, or `filter`.

NodeList : it is a collection of DOM nodes (which can be elements, text nodes, comments, etc.) that are returned by methods like **querySelectorAll**, **childNodes**.

DOM Tree:

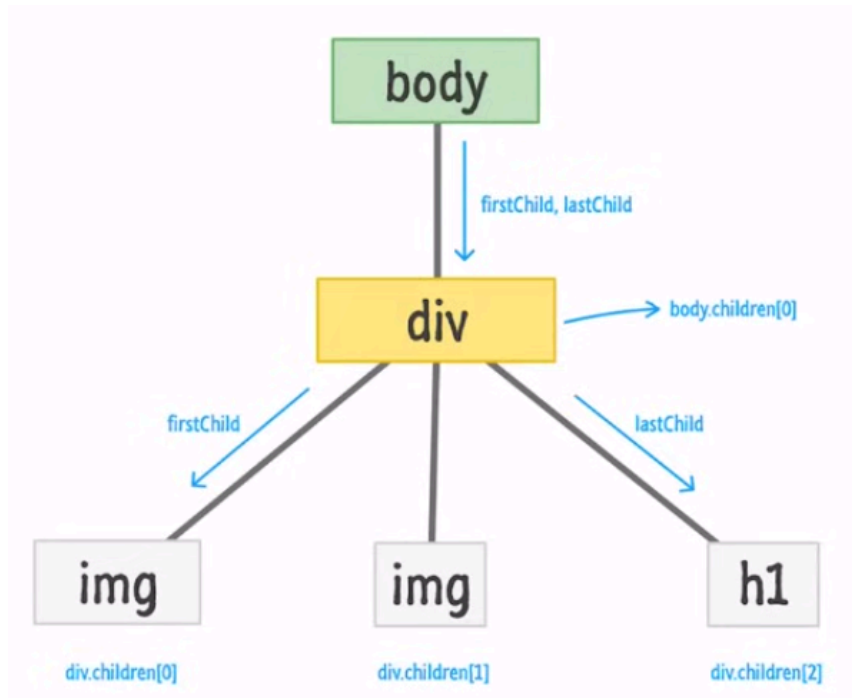
The DOM (Document Object Model) is structured as a tree of nodes, where each node represents an element, attribute, text, or comment in the document. The tree structure allows you to navigate through the document and manipulate its elements.

Nodes in the DOM:

- **Element Node (nodeType = 1)**: Represents HTML elements like `<div>`, `<p>`, etc.
- **Text Node (nodeType = 3)**: Represents the text inside an element.
- **Comment Node (nodeType = 8)**: Represents comments in the HTML.
- **Document Node (nodeType = 9)**: Represents the entire document.

JAVASCRIPT

Key Properties:



1. firstChild and lastChild:

- **firstChild:** Refers to the first child node of an element, which can be an element node, text node, or comment node.
- **lastChild:** Refers to the last child node of an element.
- **Example :**

```
const parent = document.getElementById("parent");  
console.log(parent.firstChild); // Could be a text node or element node.  
console.log(parent.lastChild); // Could be a text node or element node.
```

2. firstElementChild and lastElementChild:

- **firstElementChild:** Refers to the first child element node (ignores text and comment nodes).
- **lastElementChild:** Refers to the last child element node.
- **Example:**

```
console.log(parent.firstElementChild); // First element node (e.g., <div>, <p>).  
console.log(parent.lastElementChild); // Last element node.
```

JAVASCRIPT

3. children Property:

- **children:** Returns an HTMLCollection of all child element nodes of an element (ignores text and comment nodes).
- **Example:**
`console.log(parent.children); // HTMLCollection of all child elements.`

4. previousElementSibling :

Description: Returns the previous sibling that is an element (ignores text nodes).

Category: Element Property

Code : `let prevElem = element.previousElementSibling;`

5. previousSibling :

Description: Returns the previous sibling node (can be any node type: element, text, etc.).

Category: Node Property

Code : `let prevSibling = element.previousSibling;`

6. replaceChild(newChild, oldChild):

Description: Replaces a child node with a new node.

Category: Node Method

Code: `parent.replaceChild(newChild, oldChild);`

7. parentElement:

Description: Returns the parent element of the specified element.

Category: Element Property

Code: `let parentElem = element.parentElement;`

8. parentNode :

Description: Returns the parent node of the specified node (can be an element or other node types).

Category: Node Property

Code: `let parent = element.parentNode;`

9. nextElementSibling :

Description: Returns the next sibling that is an element (ignores text nodes).

JAVASCRIPT

Category: Element Property

Code:`let nextElem = element.nextElementSibling;`

10. **nextSibling :**

Description: Returns the next sibling node (can be any node type: element, text, etc.).

Category: Node Property

Code:`let nextSibling = element.nextSibling;`

2. **Modifying Elements:**

Once you've selected an element, you can modify its content, attributes, or styles.

Modifying Content:

You can change the inner HTML or text content of an element.

1. **tagName :**

The `tagName` property in the DOM is used to return the tag name of an element. It provides the name of the HTML or XML element in **uppercase** for HTML documents (e.g., DIV, P, SPAN, etc.).

Example :

```
const element = document.getElementById("myElement");
console.log(element.tagName);
// Output: "DIV" (if the element is a <div>)
```

JAVASCRIPT

2. **innerText** :

It returns the text content of the element and all its children.

The **innerText** property in the DOM is used to get or set the visible text content of an HTML element. It represents the text inside an element as it appears to the user, excluding any hidden elements and considering CSS styling such as `display: none` or `visibility: hidden`.

Syntax:

```
element.innerText  
element.innerText = "New text content";
```

Example :

```
<div id="example">  
  <p>This is a paragraph.</p>  
  <p style="display:none;">This is a hidden paragraph.</p>  
</div>
```

```
const element = document.getElementById("example");
```

```
//Getting the visible text content
```

```
console.log(element.innerText); Output: "This is a paragraph."
```

```
//Setting the text content
```

```
element.innerText = "Updated text content.";
```

The <div> now only contains the text "Updated text content." with no HTML tags.

Difference Between innerText and textContent:

- **innerText:** Considers CSS styling and only returns visible text. It also recognizes line breaks and formatting as it appears on the page.

3. **textContent**:

It retrieves all the text content of an element, including hidden elements, and doesn't respect the visual styling (like `display: none`). It also returns the text without formatting (e.g., no line breaks).

Example Comparison

```
const element = document.getElementById("example");
```

JAVASCRIPT

```
console.log(element.innerText); // "This is a paragraph."  
console.log(element.textContent); // "This is a paragraph.This is a  
hidden paragraph."
```

4. **innerHTML** :

The `innerHTML` property in the DOM is used to get or set the HTML content inside an element. It allows you to access or modify the entire HTML structure within a specific element, including **tags**, text, and other nested elements.

Syntax:

element.innerHTML

element.innerHTML = "New HTML content";

Example:

1. Getting HTML Content:

```
// Example HTML: <div id="content"><p>This is a paragraph.</p></div>  
const element = document.getElementById("content");  
console.log(element.innerHTML);
```

Output: "<p>This is a paragraph.</p>"

2. Setting HTML Content: set new text or element and remove old text and elements.

```
// Setting new HTML content  
element.innerHTML = "<h1>New Heading</h1><p>New  
paragraph.</p>";
```

//The <div id="content"> now contains:

```
<h1>New Heading</h1>  
<p>New paragraph.</p>
```

5. **outerHTML** :

Description: Gets or sets the HTML markup of an element, including the element itself.

JAVASCRIPT

```
let item = document.querySelector("#item-list");

console.log(item.innerText);           //item 3

console.log(item.innerHTML);          //<b>item 3</b>

console.log(item.outerHTML);          //<li id="item-list2"><b>item 3</b></li>

item.outerHTML = "<li><i>hello list item 3</i></li>";
```

Modifying Attributes:

You can get or set attributes such as src, href, class, id etc.

setAttribute and **getAttribute** methods are used to modify and retrieve attributes of HTML elements.

1. setAttribute()

The setAttribute() method is used to add a new attribute to an element or modify the value of an existing attribute.

Syntax: element.setAttribute(name, value);

element: The DOM element you want to modify.

name: The name of the attribute you want to set (e.g., id, class, href).

value: The value you want to assign to the attribute.

Example:

```
let element = document.getElementById("myElement");

element.setAttribute("class", "newClass");
```

2. getAttribute()

The getAttribute() method retrieves the value of a specified attribute from an element.

Syntax: let attributeValue = element.getAttribute(name);

element: The DOM element from which you want to retrieve the attribute.

name: The name of the attribute you want to retrieve.

JAVASCRIPT

Example:

```
let element = document.getElementById("myElement");  
  
let classValue = element.getAttribute("class");  
  
console.log(classValue);
```

3. **removeAttribute()**

The `removeAttribute()` method is used to remove an attribute from an element.

Syntax: `element.removeAttribute(name);`

name: The name of the attribute you want to remove.

Example:

```
let element = document.getElementById("myElement");  
element.removeAttribute("class");
```

4. **hasAttribute()**

The `hasAttribute()` method checks whether an element has a specified attribute.

Syntax: `let hasAttr = element.hasAttribute(name);`

name: The name of the attribute you want to check.

Example:

```
let element = document.getElementById("myElement");  
if (element.hasAttribute("class")) {  
    console.log("Element has a class attribute");  
}
```

5. **Direct Attribute Modification**

You can directly access and modify some common attributes using element properties, such as **id**, **className**, **src**, **href**, **value**, etc.

Example:

```
let element = document.getElementById("myElement");  
  
// Set attributes directly  
element.id = "newId";  
element.className = "newClass";
```

JAVASCRIPT

```
element.src = "image.jpg";  
element.href = "https://example.com";
```

// Get attribute values directly

```
let idValue = element.id;  
let classValue = element.className;  
let srcValue = element.src;  
let hrefValue = element.href;
```

6. classList Property (for Class Manipulation)

The classList property provides **methods to add, remove, toggle**, or check classes on an element.

Example:

```
let element = document.getElementById("myElement");
```

// Add a class

```
element.classList.add("newClass");
```

// Remove a class

```
element.classList.remove("oldClass");
```

// Toggle a class

```
element.classList.toggle("active");
```

// Check if the element has a class

```
if (element.classList.contains("active")) {  
    console.log("Element is active");  
}
```

Toggle Active State

In the context of web development, "toggling" an active state typically refers to switching an element's state between "active" and "inactive" by adding or removing an **active** class (or any similar class name) from its **classList**. This technique is commonly used for interactive elements like buttons, menu items, or tabs to indicate which item is currently selected or active.

Example Scenario: Toggle Active State

JAVASCRIPT

Imagine you have a navigation menu, and you want to highlight the currently selected menu item. You can do this by toggling an **active** class on the clicked menu item.

HTML Structure:

```
<ul>
  <li id="home" class="nav-item">Home</li>
  <li id="about" class="nav-item">About</li>
  <li id="contact" class="nav-item">Contact</li>
</ul>
```

CSS:

```
.nav-item {
  padding: 10px;
  cursor: pointer;
}
.nav-item.active {
  background-color: blue;
  color: white;
}
```

JavaScript to Toggle Active State:

```
let navItems = document.querySelectorAll('.nav-item');
navItems.forEach(item => {
  item.addEventListener('click', function() {
    // Remove 'active' class from all items
    navItems.forEach(nav => nav.classList.remove('active'));
    // Add 'active' class to the clicked item
    this.classList.add('active');
  });
});
```

Toggle with `classList.toggle()`

If you want to simply toggle the active state on a single element (e.g., a button that turns on/off), you can use **`classList.toggle()`**:

Example:

```
<!-- Button element -->
<button id="toggleButton">Click Me</button>

<!-- JavaScript to handle the toggle -->
<script>
```

JAVASCRIPT

```
let button = document.getElementById("toggleButton");

button.addEventListener("click", function () {
  this.classList.toggle("inactive");

  if (this.classList.contains("active")) {
    console.log("Button is active");
  } else {
    console.log("Button is inactive");
  }
});
</script>
```

Modifying Styles:

You can directly apply inline CSS styles to elements.

Modifying styles of HTML elements using the DOM can be done in two primary ways: directly manipulating the style attribute of an element or by adding/removing classes that have associated CSS styles. Here's a detailed look at both approaches:

1.Modifying Styles Using the style Attribute

The **style attribute** in the DOM allows you to directly set CSS properties for an element.

Accessing the style Attribute

You can access the style attribute of an element via the style property in JavaScript, which represents the inline styles of that element.

Syntax: `element.style.propertyName = value;`

propertyName: The CSS property you want to modify (in **camelCase**, e.g., **backgroundColor** for background-color).

value: The value you want to set for that property.

Example:

```
let element = document.getElementById("myElement");
```

JAVASCRIPT

// Set individual styles

```
element.style.color = "red";  
element.style.backgroundColor = "blue";  
element.style.fontSize = "20px";
```

// Remove an inline style

```
element.style.color = ""; // This removes the inline color style
```

2. Modifying Styles Using the className or classList

Instead of directly modifying the style attribute, you can also modify the classes assigned to an element, which can control its styles via CSS.

1. Using className

className allows you to replace the entire set of classes on an element.

Syntax: `element.className = "newClassName";`

Example:

```
let element = document.getElementById("myElement");
```

// Set a new class

```
element.className = "highlighted";
```

This approach is simple but replaces all existing classes, which can be limiting.

2. Using classList

classList provides a more flexible way to work with classes.

Syntax: `element.classList.add("newClassName");`

Example :

```
let element = document.getElementById("myElement");
```

// Add a class

```
element.classList.add("newClass");
```

JAVASCRIPT

// Remove a class

```
element.classList.remove("newClass");
```

This method is preferable when you need to add or remove specific classes without affecting others.

3. Creating and Inserting Elements:

You can create new elements and insert them into the DOM at specific locations.

Creating Elements:

You can create a new HTML element using **document.createElement**.

The `createElement()` method in JavaScript is used to create a new HTML element dynamically. This allows you to add new elements to the DOM (Document Object Model) on the fly, without needing to write them directly into the HTML code.

How It Works:

Create the Element:

- You specify the type of element you want to create (like a `div`, `p`, `button`, etc.) using `createElement()`.

Customize the Element:

- After creating the element, you can set its attributes, styles, classes, and content.

Add the Element to the DOM:

- Once the element is ready, you can insert it into the document by appending it to an existing parent element using methods like `appendChild()` or `append()`.

Syntax:

- **document.createElement(tagName);**
- **tagName:** The type of element you want to create (e.g., `"div"`, `"p"`, `"span"`).

JAVASCRIPT

Example: Basic Usage

Let's say you want to create a new paragraph element and add it to a div with the ID content.

```
<div id="content"></div>
```

JavaScript Code:

```
// Create a new paragraph element
```

```
const newParagraph = document.createElement("p");
```

```
// Add some text to the paragraph
```

```
newParagraph.textContent = "This is a dynamically created paragraph.";
```

```
// Find the parent element (the div with the ID 'content')
```

```
const parentDiv = document.getElementById("content");
```

```
// Add the new paragraph to the parent div
```

```
parentDiv.appendChild(newParagraph);
```

Result:

```
<div id="content">
```

```
  <p>This is a dynamically created paragraph.</p>
```

```
</div>
```

Inserting Elements:

You can insert the new element into the DOM using methods like **append()**, **prepend()**, **after()**, and **before()** or **appendChild**, **insertBefore**.

1. **append():**

Purpose: Inserts a node (or multiple nodes) as the last child of a parent element.

Usage: Adds content inside the selected element, at the end.

Example :

```
let parent = document.getElementById('parentElement');
```

```
let newElement = document.createElement('div');
```

```
newElement.textContent = 'Appended Element';
```

```
parent.appendChild(newElement); // Adds newElement as the last child of  
parentElement
```

JAVASCRIPT

2. prepend():

Purpose: Inserts a node (or multiple nodes) as the first child of a parent element.

Usage: Adds content inside the selected element, at the beginning.

Example :

```
let parent = document.getElementById('parentElement');
let newElement = document.createElement('div');
newElement.textContent = 'Prepended Element';
parent.prepend(newElement);
// Adds newElement as the first child of parentElement
```

3. after():

Purpose: Inserts a node (or multiple nodes) immediately after the selected element.

Usage: Adds content outside the selected element, directly after it in the DOM.

Example :

```
let reference = document.getElementById('referenceElement');
let newElement = document.createElement('div');
newElement.textContent = 'Element After';
reference.after(newElement); // Inserts newElement right after
referenceElement
```

4. before():

Purpose: Inserts a node (or multiple nodes) immediately before the selected element.

Usage: Adds content outside the selected element, directly before it in the DOM.

Example :

```
let reference = document.getElementById('referenceElement');
let newElement = document.createElement('div');
newElement.textContent = 'Element Before';
reference.before(newElement); // Inserts newElement right before
referenceElement
```

JAVASCRIPT

5. **appendChild():**

The **appendChild()** method is used to add a new element or node as the last child of a parent element.

Imagine you have a list or a box, and you want to add something new to the end of it. `appendChild()` helps you do that by placing the new item at the very end.

Example :

```
// Create a new list item
const newItem = document.createElement("li");
newItem.textContent = "New item";

// Find the list where you want to add the item
const list = document.getElementById("myList");

// Add the new item to the end of the list
list.appendChild(newItem);
```

6. **insertBefore()**

The **insertBefore()** method is used to insert a new element or node before a specific existing child node in a parent element.

If you want to add something in the middle of a list or before a specific item, `insertBefore()` lets you place it exactly where you want.

Example: firstChild and lastChild

```
// Create a new list item

const newItem = document.createElement("li");
newItem.textContent = "New item";

// Find the list and the item you want to insert before
const list = document.getElementById("myList");
const firstItem = list.firstChild; // Assume you want to insert before the first item

// Insert the new item before the first item
list.insertBefore(newItem, firstItem);
```

Example: Inserting at a Specific Position

JAVASCRIPT

```
//Get the third item (index 2, because indexes start at 0)
const thirdItem = list.children[2];
// Insert the new item before the third item
list.insertBefore(newItem, thirdItem);
```

Example for Middle Insertion:

```
// Calculate the middle index
const middleIndex = Math.floor(list.children.length / 2);

// Get the middle item
const middleItem = list.children[middleIndex];

// Insert the new item before the middle item
list.insertBefore(newItem, middleItem);
```

insertBefore() requires two things: the new node you want to add and the existing node you want to insert it before.

7. insertAdjacentHTML(position, text)

Purpose: Inserts an HTML string at a specified position relative to an element.

Use Case: When you want to insert HTML content directly, including any HTML tags.

Example

```
const main_div = document.querySelector(".container");
main_div.insertAdjacentHTML("beforeend", "<p>New paragraph</p>");
```

Positions : beforebegin,afterbegin,beforeend,afterend

8. insertAdjacentElement(position, element)

Purpose: Inserts an existing DOM element at a specified position relative to another element.

Use Case: When you need to insert a DOM element (that you've created or selected) in relation to another element.

Example

```
const main_div = document.querySelector(".container");
const newElement = document.createElement("p");
newElement.textContent = "New paragraph";
main_div.insertAdjacentElement("beforeend", newElement);
```

JAVASCRIPT

Positions : beforebegin,afterbegin,beforeend,afterend

9. insertAdjacentText(position, text)

Purpose: Inserts plain text at a specified position relative to an element.

Use Case: When you need to insert plain text without any HTML tags.

Example:

```
const main_div = document.querySelector(".container");  
main_div.insertAdjacentText("beforeend", "Just some text");
```

Positions : beforebegin,afterbegin,beforeend,afterend

Delete Elements:

To delete elements in the DOM using JavaScript :

1. remove()

Purpose: Directly removes the selected element from the DOM.

Usage: Simple and straightforward, just call `remove()` on the element you want to delete.

```
let element = document.getElementById('elementToRemove');  
element.remove(); // Removes the element with the ID  
'elementToRemove' from the DOM
```

2. removeChild()

Purpose: Removes a child element from its parent node.

Usage: You need to call this method on the parent element and pass the child element you want to remove.

Example:

```
let parent = document.getElementById('parentElement');  
let child = document.getElementById('childElement');  
parent.removeChild(child); // Removes 'childElement' from  
'parentElement'
```

3. innerHTML (Clearing Content)

JAVASCRIPT

Purpose: Removes all child elements by clearing the HTML content of a parent element.

Usage: Set **innerHTML** of the parent element to an empty string

Example :

```
let parent = document.getElementById('parentElement');
```

```
parent.innerHTML = ""; // Removes all child elements inside  
'parentElement'
```

26. Events in js

What are Events in JavaScript?

Events in JavaScript are actions or occurrences that happen on a webpage, triggered by user interactions (like clicks or key presses) or by the browser itself (like when the page finishes loading).

The Change in the state of an object is known as an event.

Why Do We Use Events in JavaScript?

Events make web pages interactive by allowing us to run JavaScript code in response to specific actions. For example, when a user clicks a button, we can trigger a function to show a message, submit a form, or update content.

Types of JavaScript Events

JavaScript events are broadly categorized based on the type of user interaction or browser action that triggers them. Here are some common types:

Mouse Events:

1. click:

Triggered when an element is clicked.

JAVASCRIPT

Example : Inline event handling in html

```
<button id="btn1" onclick="console.log('This is on click event')">  
    CLICK ME 1  
</button>
```

Example : event handling in js

```
const btn1 = document.getElementById("btn1");  
btn1.onclick = () => {  
    console.log("button 1 was clicked");  
    alert("hello i am button 1");  
};
```

2. dblclick:

Triggered when an element is double-clicked.

Example : Inline event handling in html

```
<button id="btn2" ondblclick="console.log('This is dbl - click event')">  
    CLICK ME 2  
</button>
```

Example : event handling in js

```
const btn2 = document.getElementById("btn1");  
btn2.ondblclick = () => {  
    console.log("button 2 was clicked");  
    alert("hello i am button 2");  
};
```

3. mouseover:

Triggered when the mouse pointer is moved over an element.

Example : Inline event handling in html

```
<div onmouseover="style.backgroundColor = 'blue'"> I am box</div>
```

Example : event handling in js

```
const box = document.querySelector("div");
```

```
box.onmouseover = () => {
```

JAVASCRIPT

```
console.log("hello I'm a box");  
box.style.backgroundColor = "black";  
};
```

4. mouseout:

Triggered when the mouse pointer is moved out of an element.

Example : Inline event handling in html

```
<div id="box" onmouseout="style.backgroundColor = 'red'">  
I am box</div>
```

5. mousemove:

Triggered when the mouse is moved within an element.

Example : Inline event handling in html

```
<div id="box" onmousemove="style.backgroundColor = 'orange'">I am  
box</div>
```

Event Object

It is a special object that has details about the event.

All event handlers have access to the Event Object's properties and methods.

The event object in JavaScript is a special object that gets automatically passed to an event handler when an event occurs. It contains details about the event, such as the type of event, the element that triggered it, and the mouse position or key pressed, allowing you to access and manipulate event-related information within your code.

Syntax :

```
Node.event = (e) =>{  
//handle here  
}
```

Properties and methods : e.target, e.type, e.clientX, e.clientY.

Example :

```
<button id="btn" onclick="console.log('event object in js');"> CLICK ME  
</button>
```

JAVASCRIPT

```
const btn = document.querySelector("#btn");

btn.onclick = (e) => {
  console.log("event object");
  console.log(e);
  console.log(e.target);
  console.log(e.type);
};
```

Event Listeners

What Are Event Listeners in JavaScript?

Event listeners are methods in JavaScript that allow you to "listen" for specific events on an element, such as clicks, key presses, or mouse movements. When the event occurs, the listener triggers a corresponding event handler function.

How to Use Event Listeners

You attach an event listener to an element using the `addEventListener()` method. This method takes two main arguments: the event type (e.g., "click", "keyup") and the function that should be executed when that event occurs.

Syntax:

```
node.addEventListener(event , callback)
```

Example of an Event Listener

```
const button = document.getElementById("myButton");
```

```
button.addEventListener("click", function(e) {
  alert("Button was clicked!");
});
```

Explanation:

addEventListener(): This method is used to attach an event listener to the button element.

Event Type: "click" specifies that the listener is waiting for a click event.

JAVASCRIPT

Event Handler: The anonymous function `function() { ... }` runs when the click event occurs, showing an alert.

Benefits of Event Listeners

- **Separation of Concerns:** Keeps your HTML and JavaScript separate, making your code cleaner.
- **Multiple Listeners:** You can attach multiple listeners for different events or even the same event on a single element.
- **Dynamic Binding:** You can add or remove listeners dynamically in your code, allowing for flexible interactivity.

Example : Adding event Listener & Multiple Listeners :

//HTML CODE

```
<button class="my_btn">click me event</button>
```

//JS CODE

```
const btn = document.querySelector(".my_btn");
```

```
btn.addEventListener("click", () => {  
  console.log("button with event listener");  
  btn.style.backgroundColor = "red";  
});
```

```
btn.addEventListener("dblclick", () => {  
  alert("event listener");  
});
```

Example : removing event Listener :

Syntax `node.removeEventListener(event ,callback)`

Note* : the callback reference should be the same to remove.

```
const remove_id = () => {  
  alert("event listener");  
};  
btn.addEventListener("dblclick", remove_id);
```

```
btn.removeEventListener("dblclick", remove_id);
```

Example : We can also pass an event object in the over event listener.

JAVASCRIPT

```
btn.addEventListener("dblclick", (e) => {  
  console.log(e);  
});
```

Keyboard Events:

1. **keydown**: Triggered when a key is pressed down.
2. **keyup**: Triggered when a key is released.
3. **keypress**: Triggered when a key is pressed (deprecated; use keydown or keyup instead).

Example:

Html code :

```
<input type="text" id="inputField" placeholder="Type something..."  
style="width: 200px; padding: 5px;">
```

```
<p id="output"></p>
```

```
<script>
```

```
const inputField = document.getElementById("inputField");
```

```
const output = document.getElementById("output");  
inputField.addEventListener("keydown", () => {
```

```
  output.textContent = "Key down!";
```

```
});
```

```
inputField.addEventListener("keyup", () => {
```

```
  output.textContent = "Key up!";
```

```
});
```

```
inputField.addEventListener("keypress", () => {
```

```
  output.textContent = "Key press!"; });
```

```
</script>
```

JAVASCRIPT

Form Events:

submit: Triggered when a form is submitted.

change: Triggered when the value of an input field changes (like selecting a different option in a dropdown).

focus: Triggered when an element (like an input field) gains focus.

blur: Triggered when an element loses focus.

Example:

Here's an example demonstrating the submit, change, focus, and blur events:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Form Events Example</title>
</head>
<body>

  <form id="myForm">
    <label for="name">Name:</label>
    <input type="text" id="name" name="name"><br><br>

    <label for="color">Favorite Color:</label>
    <select id="color" name="color">
      <option value="">Select a color</option>
      <option value="red">Red</option>
      <option value="green">Green</option>
      <option value="blue">Blue</option>
    </select><br><br>

    <button type="submit">Submit</button>
  </form>

  <p id="output"></p>

  <script>
    const form = document.getElementById("myForm");
```

JAVASCRIPT

```
const nameInput = document.getElementById("name");  
const colorSelect = document.getElementById("color");  
const output = document.getElementById("output");
```

// submit event

```
form.addEventListener("submit", function(event) {  
    event.preventDefault(); // Prevents form from actually submitting  
    output.textContent = "Form submitted!";  
});
```

// change event

```
colorSelect.addEventListener("change", function() {  
    output.textContent = "Color changed to " + colorSelect.value + "!";  
});
```

// focus event

```
nameInput.addEventListener("focus", function() {  
    output.textContent = "Name input field focused!";  
});
```

// blur event

```
nameInput.addEventListener("blur", function() {  
    output.textContent = "Name input field lost focus!";  
});
```

```
</script>
```

```
</body>
```

```
</html>
```

Window Events:

- **load**: Triggered when the entire page has loaded.
- **resize**: Triggered when the browser window is resized.
- **scroll**: Triggered when the user scrolls within an element or the window.

Example:

Here's an example demonstrating the load, resize, and scroll window events:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

JAVASCRIPT

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Window Events Example</title>
</head>
<body style="height: 2000px;"> <!-- Extra height to enable scrolling -->

<h1>Window Events Example</h1>
<p id="output">Resize the window or scroll down to see events in action.</p>

<script>
  const output = document.getElementById("output");

  // load event
  window.addEventListener("load", function() {
    output.textContent = "Page has fully loaded!";
  });

  // resize event
  window.addEventListener("resize", function() {
    output.textContent = "Window resized to " + window.innerWidth + "x" +
window.innerHeight;
  });

  // scroll event
  window.addEventListener("scroll", function() {
    output.textContent = "Page is being scrolled! Scroll position: " +
window.scrollY + "px";
  });
</script>

</body>
</html>
```